

Introduction to Computer Vision



Lec.03: Image processing 2022.09.26

Slides adapted from Stanford CS248

Today

- **Image processing basics.**
- Image sampling.
- Image magnification.

Example image processing operations

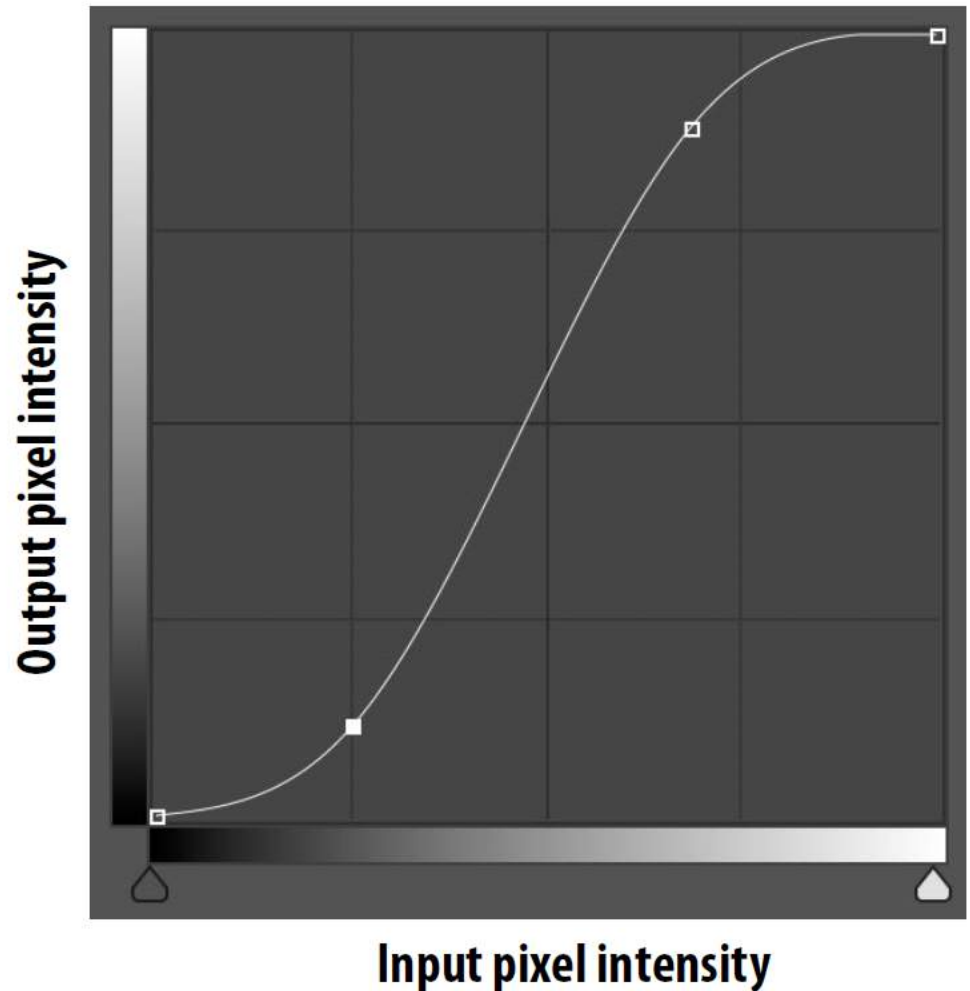


Increase contrast

Increasing contrast with “S curve”

Per-pixel operation:

$$\text{output}(x,y) = f(\text{input}(x,y))$$



Example image processing operations

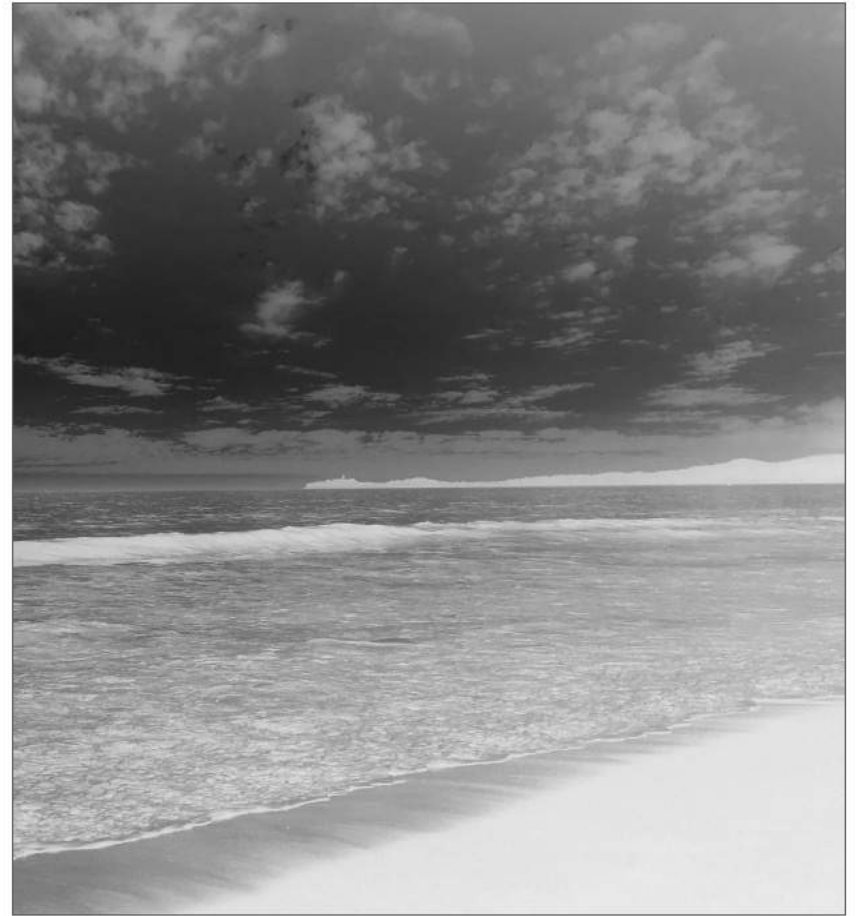
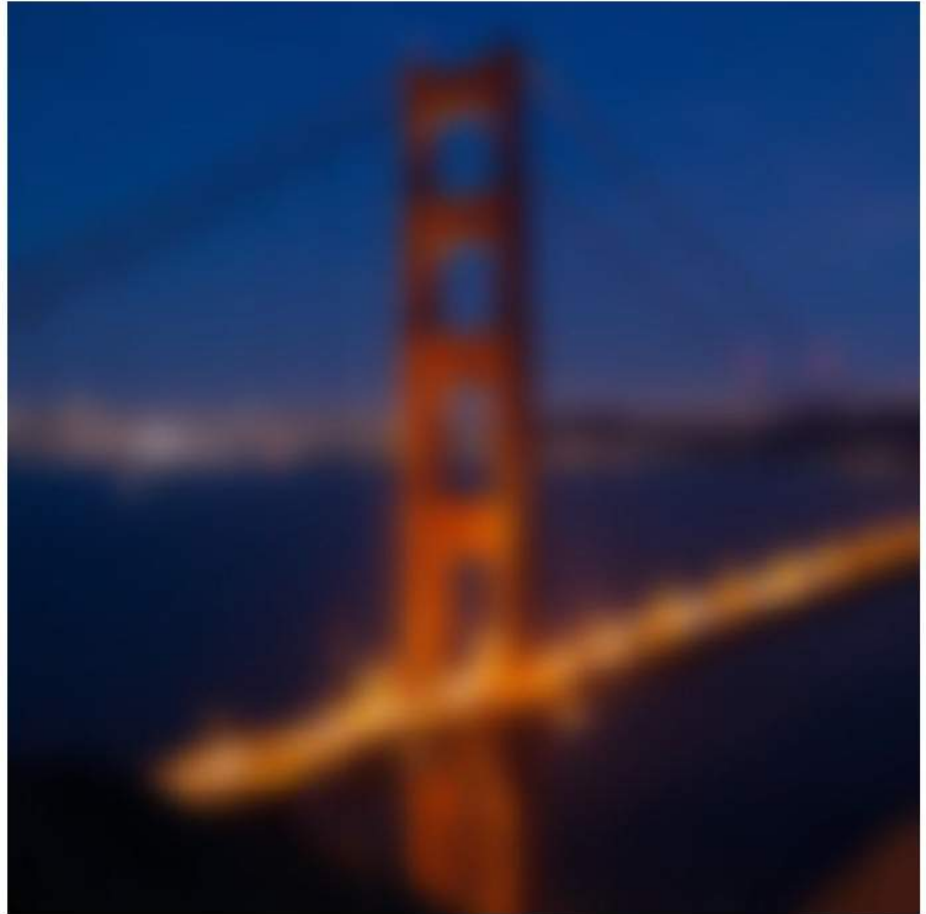


Image Invert:

$$\text{out}(x,y) = 1 - \text{in}(x,y)$$

Example image processing operations



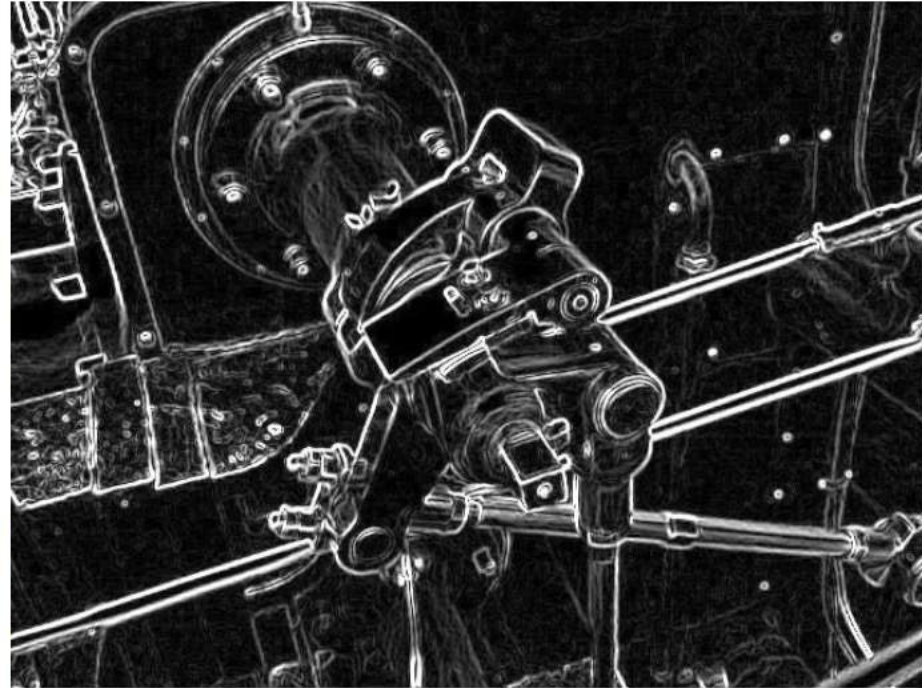
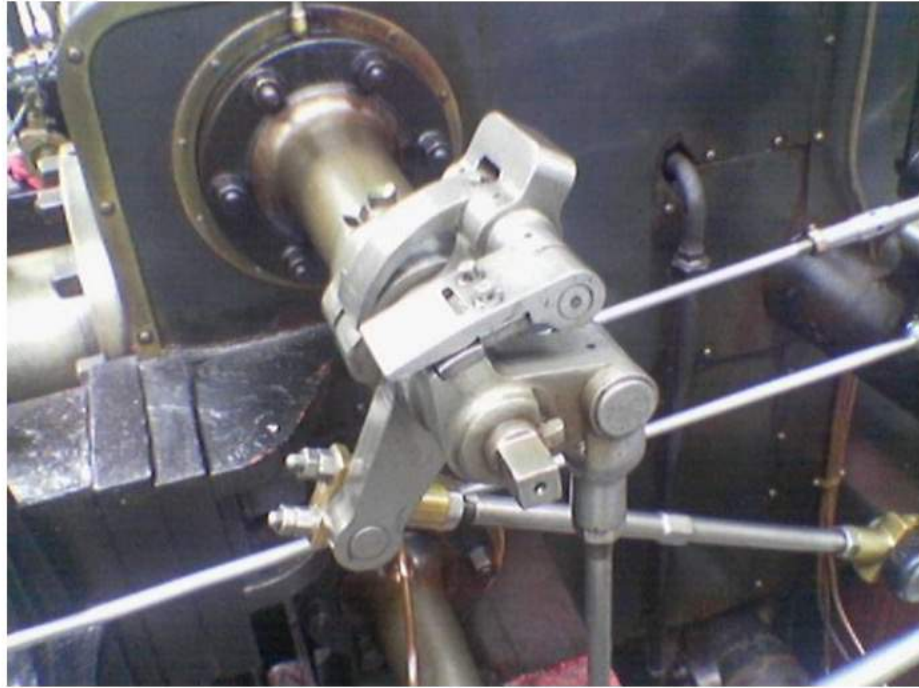
Blur

Example image processing operations



Sharpen

Edge detection



A “smarter” blur (doesn't blur over edges)



Review: convolution

$$(f * g)(x) = \int_{-\infty}^{\infty} f(y)g(x - y)dy$$

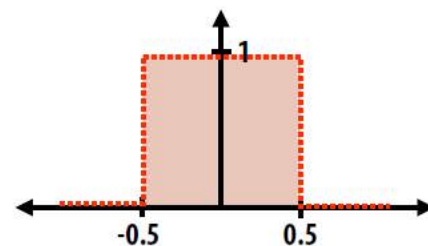
Diagram illustrating the convolution equation with red annotations:

- output signal**: points to $(f * g)(x)$
- filter**: points to $f(y)$
- input signal (e.g. the input image)**: points to $g(x - y)$

It may be helpful to consider the effect of convolution with the simple unit-area “box” function:

$$f(x) = \begin{cases} 1 & |x| \leq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

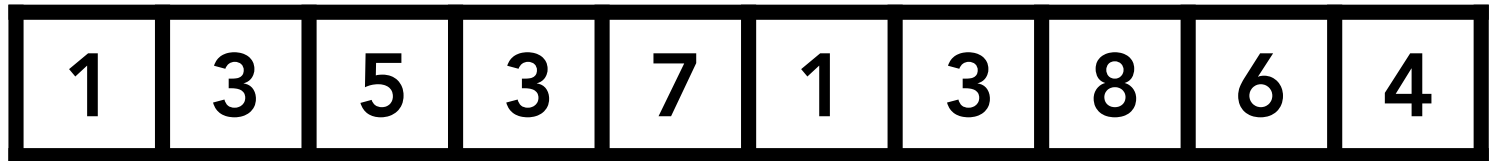
$$(f * g)(x) = \int_{-0.5}^{0.5} g(x - y)dy$$



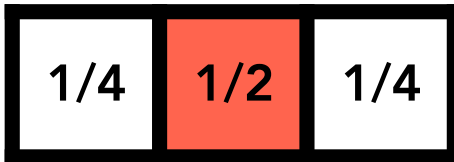
$f * g$ is a “blurred” version of g where the output at x is the average value of the input between $x-0.5$ to $x+0.5$

Convolution

Signal



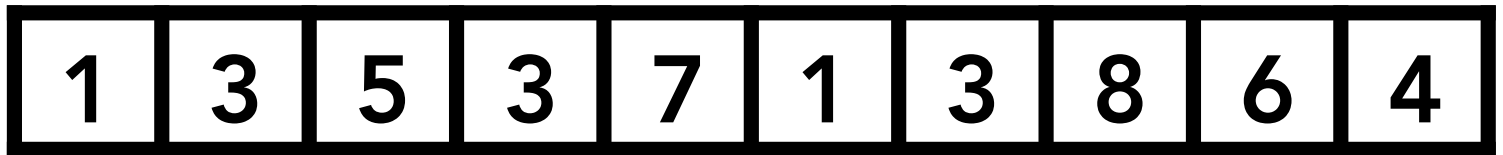
Filter



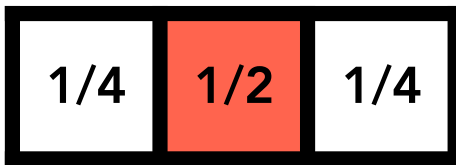
Point-wise local averaging in a “sliding window”

Convolution

Signal

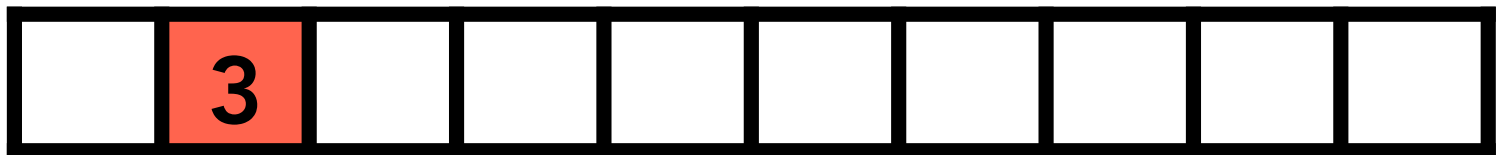


Filter



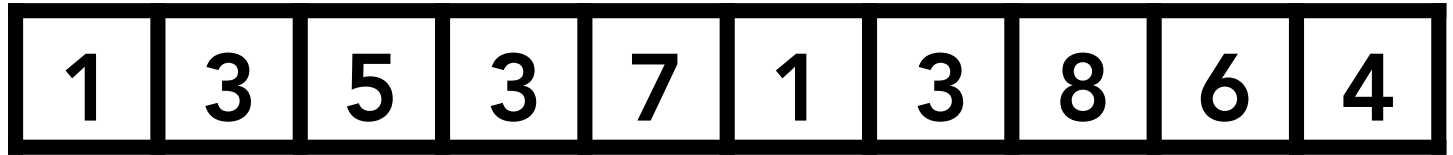
$$1 \times (1/4) + 3 \times (1/2) + 5 \times (1/4) = 3$$

Result

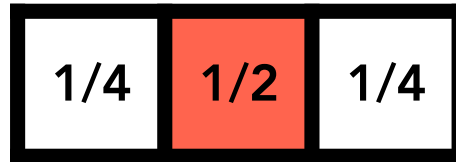


Convolution

Signal

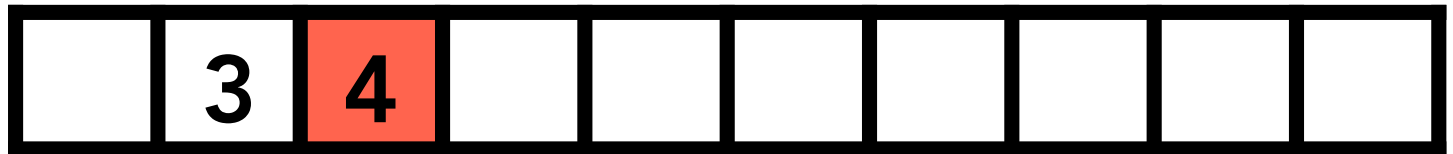


Filter

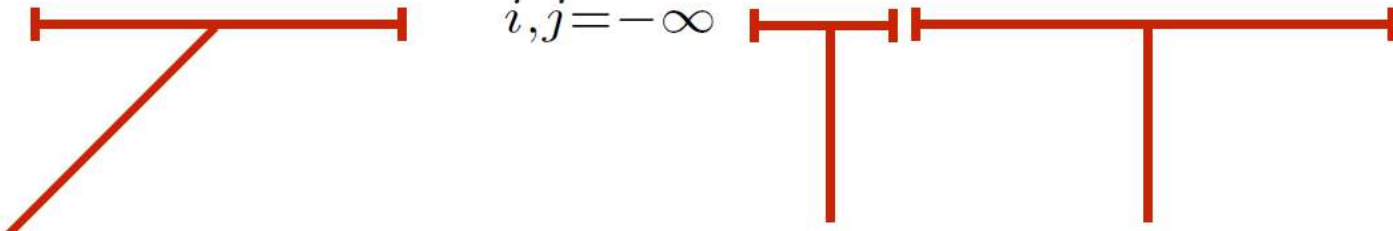


$$3 \times (1/4) + 5 \times (1/2) + 3 \times (1/4) = 4$$

Result



Discrete 2D convolution

$$(f * g)(x, y) = \sum_{i, j = -\infty}^{\infty} f(i, j) I(x - i, y - j)$$


output image

filter

input image

Consider $f(i, j)$ that is nonzero only when: $-1 \leq i, j \leq 1$

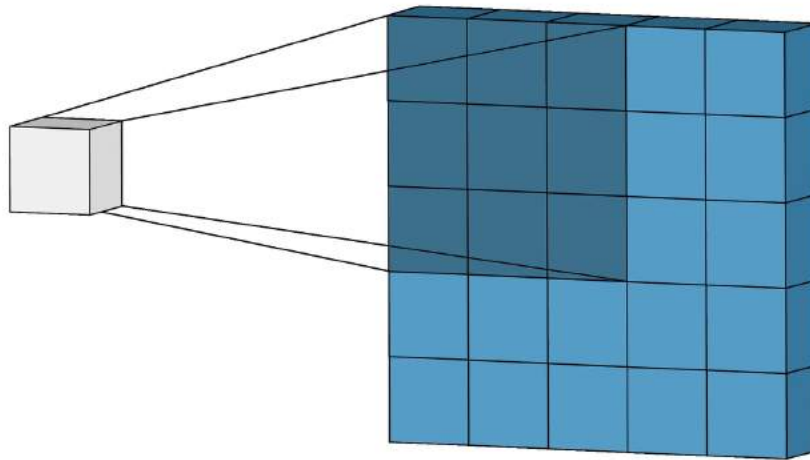
Then:

$$(f * I)(x, y) = \sum_{i, j = -1}^1 f(i, j) I(x - i, y - j)$$

And we can represent $f(i, j)$ as a 3x3 matrix of values where:

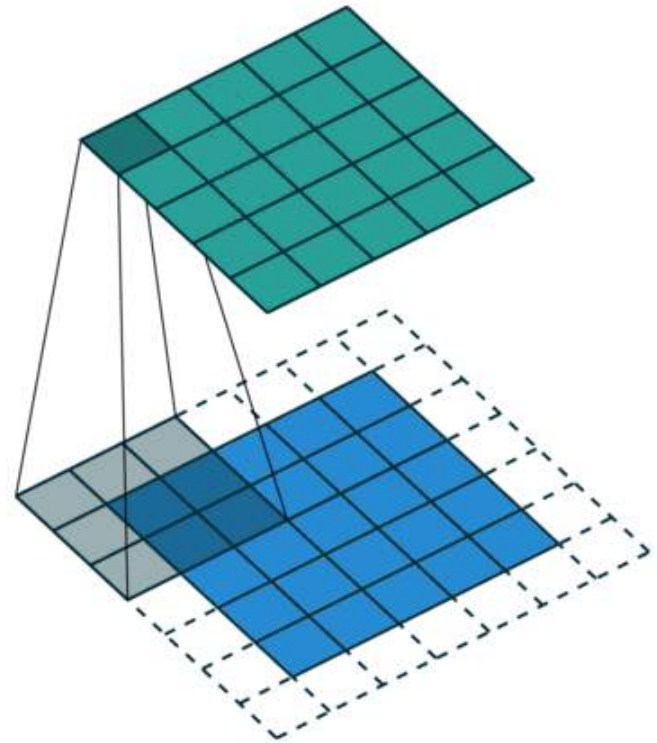
$$f(i, j) = \mathbf{F}_{i, j} \quad \text{(often called: "filter weights", "filter kernel")}$$

Discrete 2D convolution



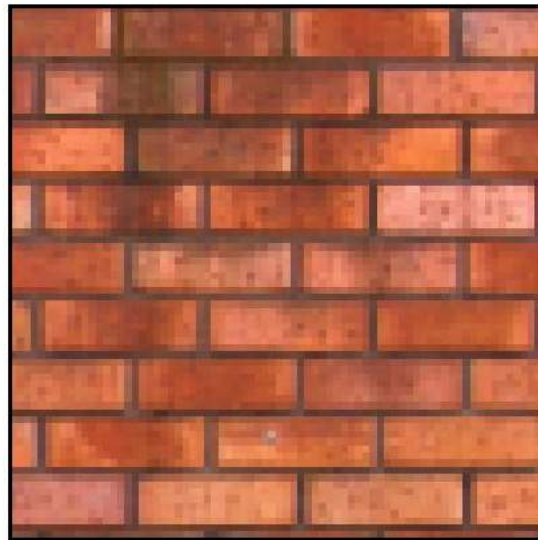
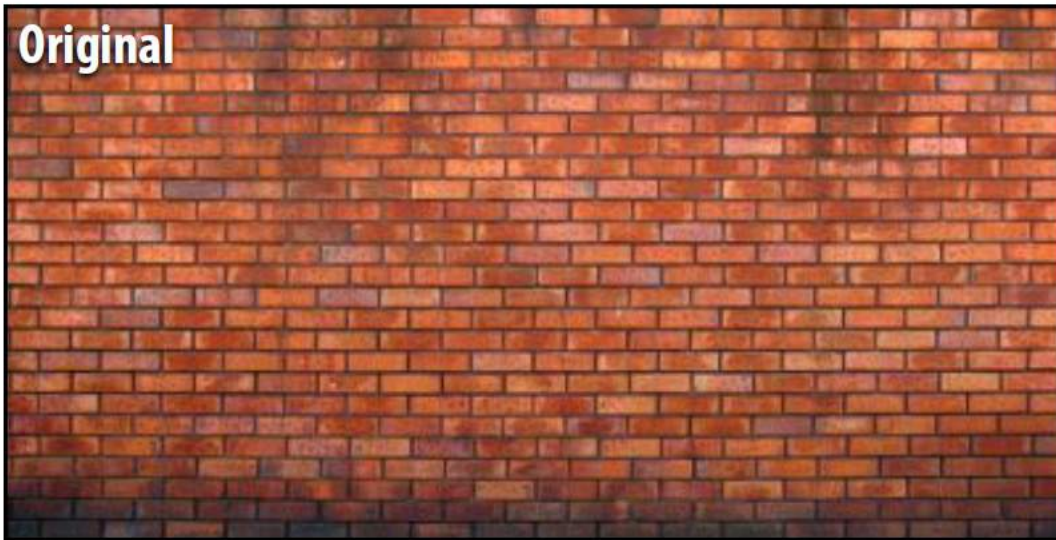
Padding

- Adding pixels around the image border
 - Zero values
 - Edge values
 - Symmetric
 - ...

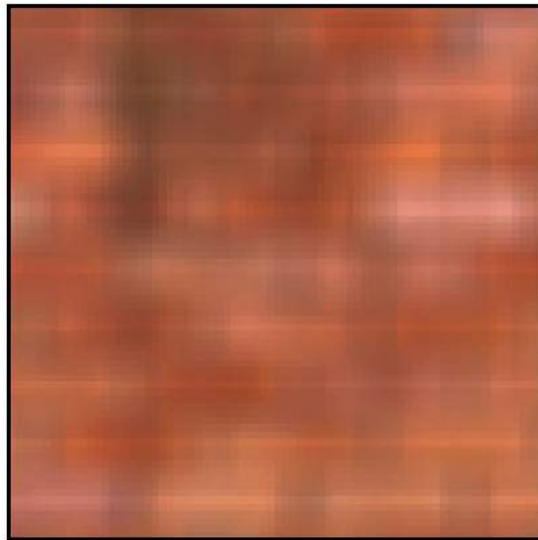
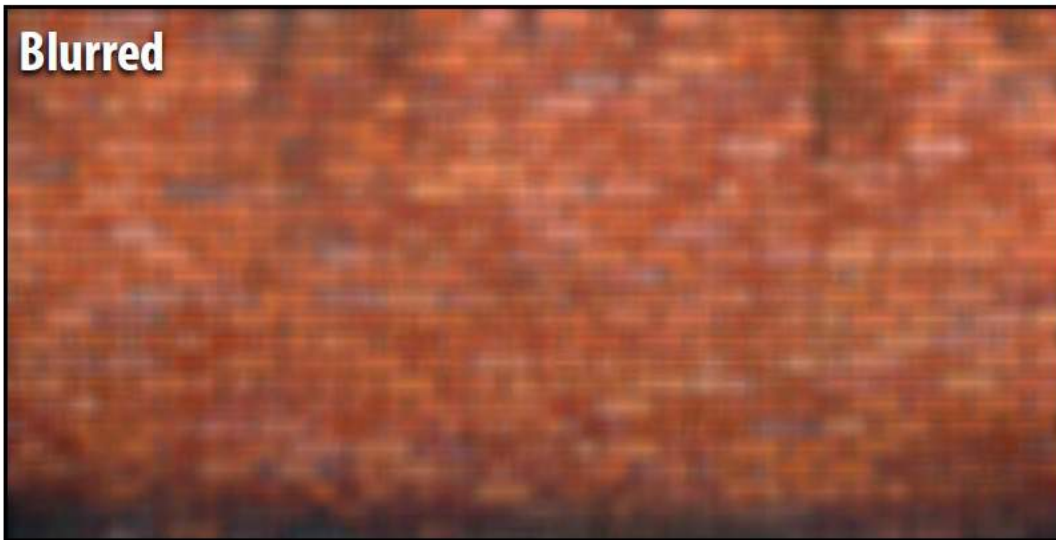


7x7 box blur

Original



Blurred



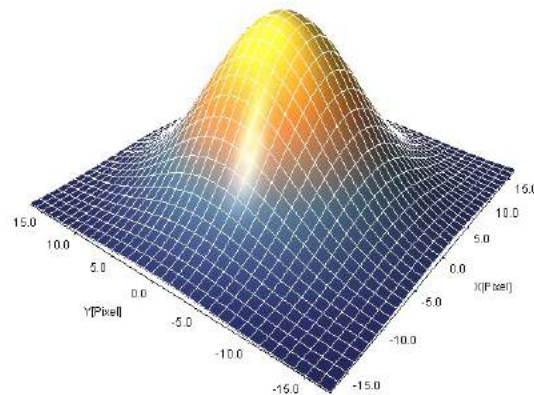
Gaussian blur

- Obtain filter coefficients by sampling 2D Gaussian function

$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2 + j^2}{2\sigma^2}}$$

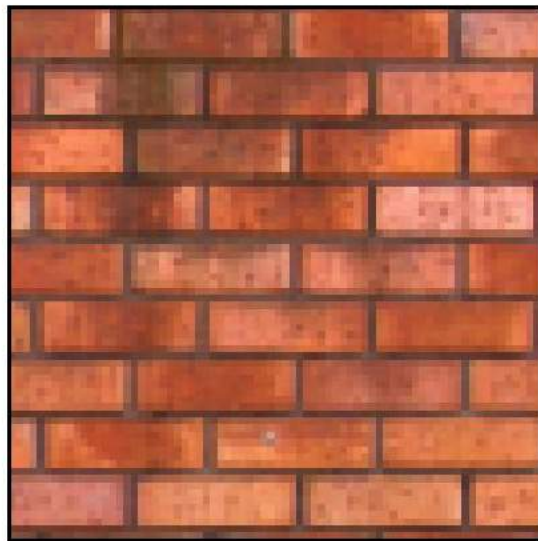
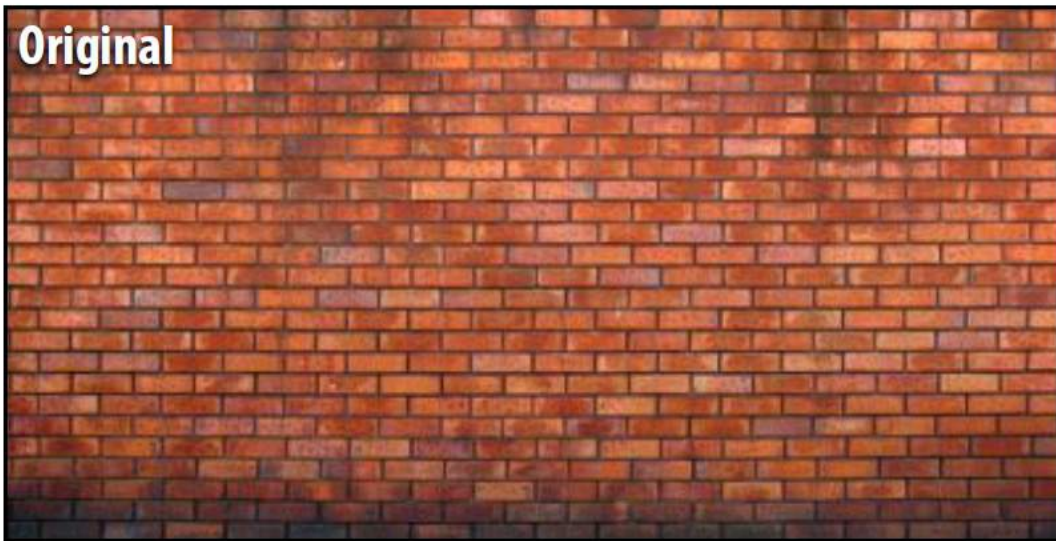
- Produces weighted sum of neighboring pixels (contribution falls off with distance)
 - In practice: truncate filter beyond certain distance for efficiency

$$\begin{bmatrix} .075 & .124 & .075 \\ .124 & .204 & .124 \\ .075 & .124 & .075 \end{bmatrix}$$



7x7 gaussian blur

Original



Blurred



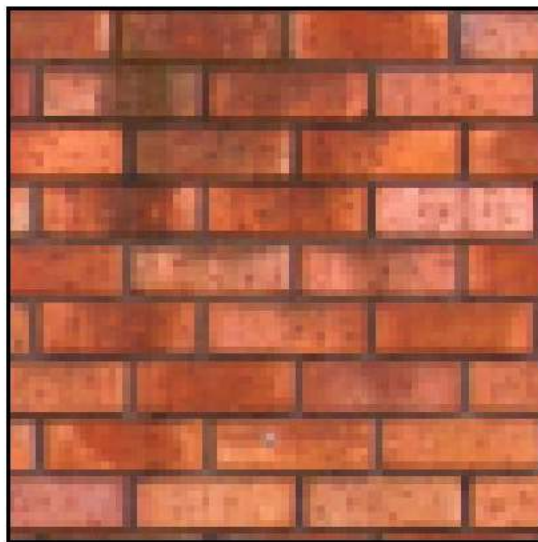
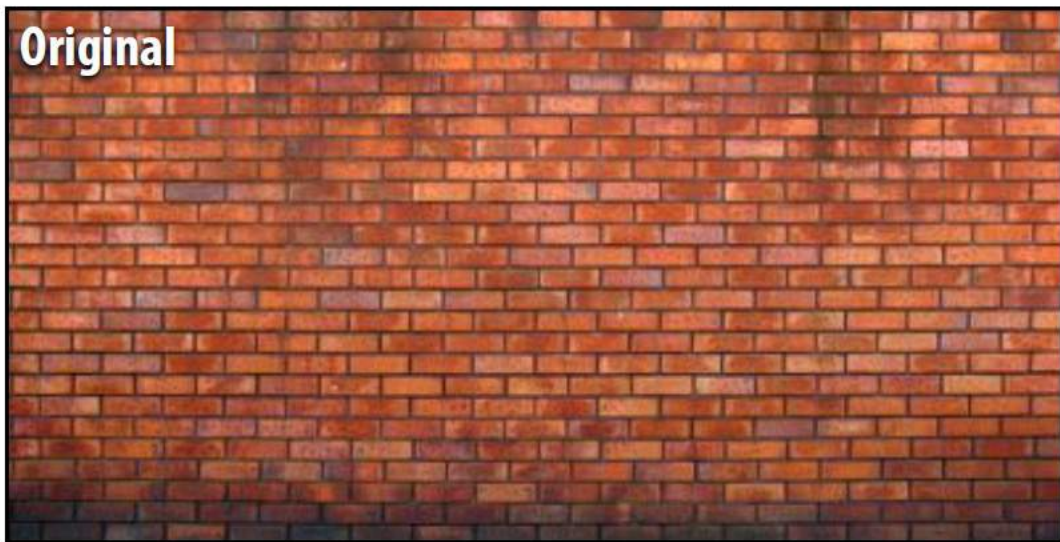
What does convolution with this filter do?

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

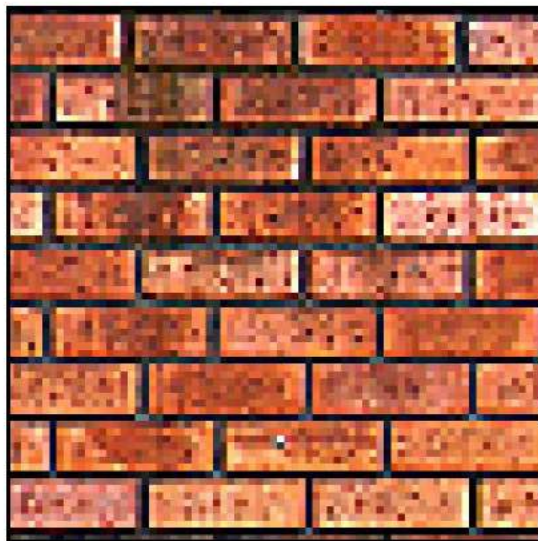
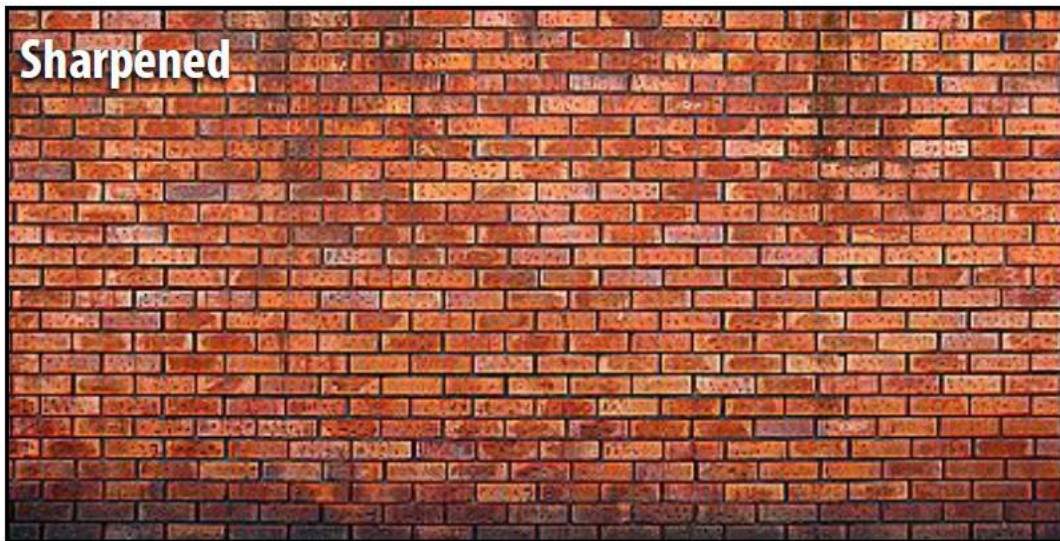
Sharpens image!

3x3 sharpen filter

Original

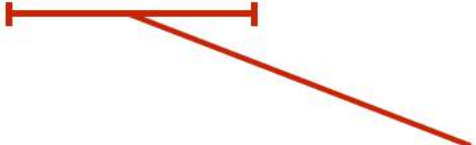


Sharpened



Sharpening is adding high frequencies

- Let I be the original image
- High frequencies in image $I = I - \text{blur}(I)$
- Sharpened image = $I + (I - \text{blur}(I))$



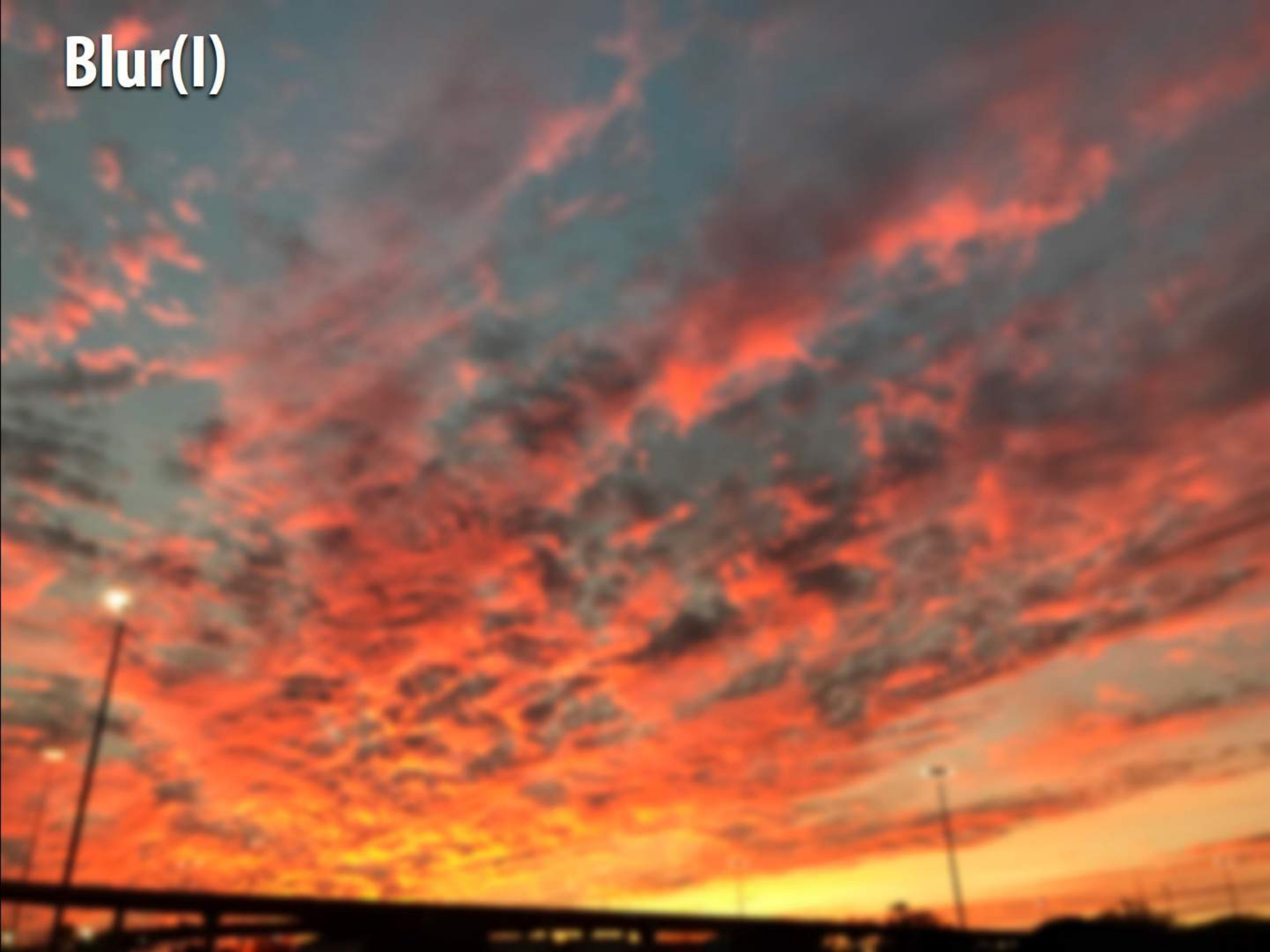
"Add high frequency content"

Original image (I)

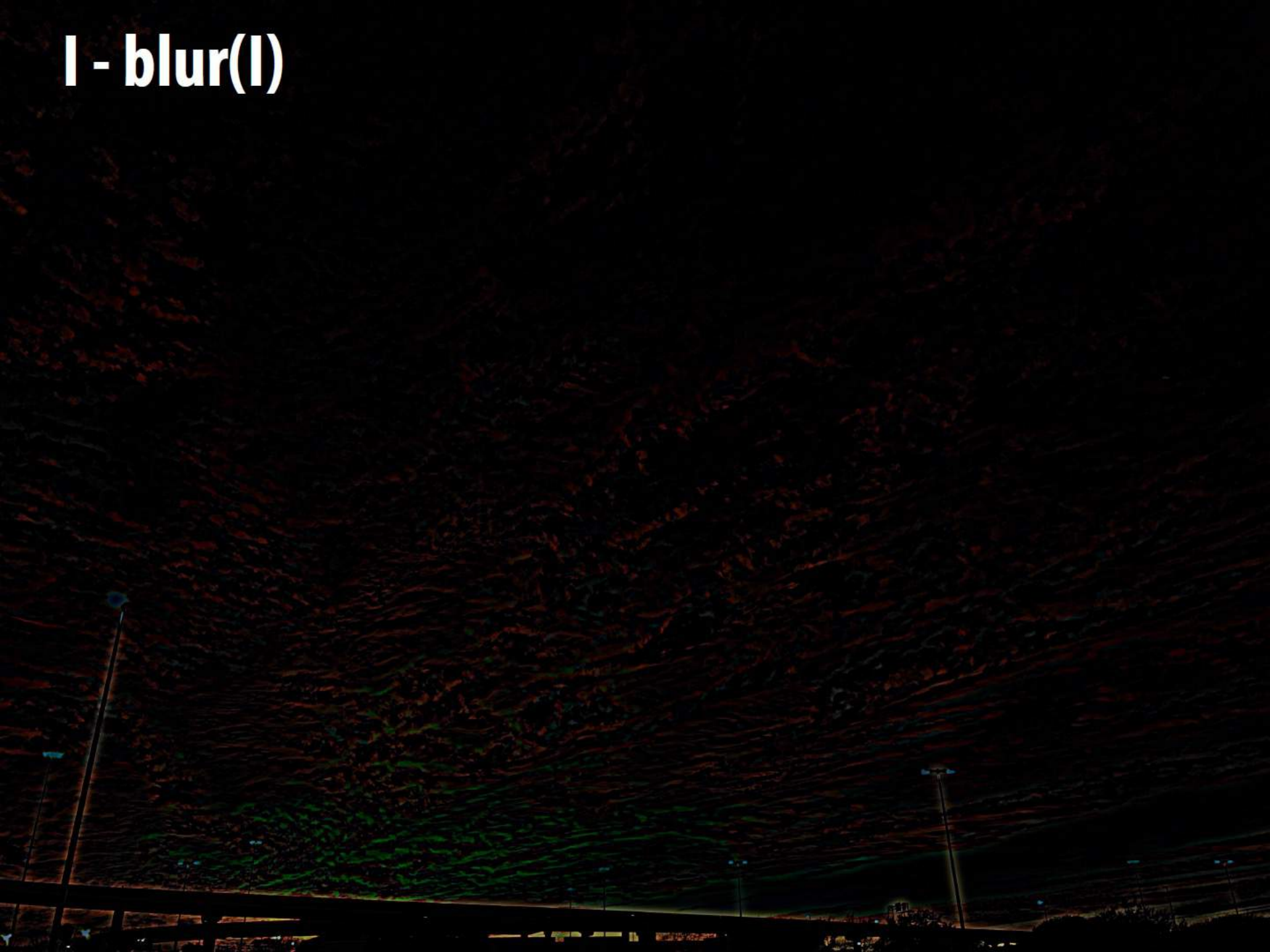
Image credit: Kayvon's parents



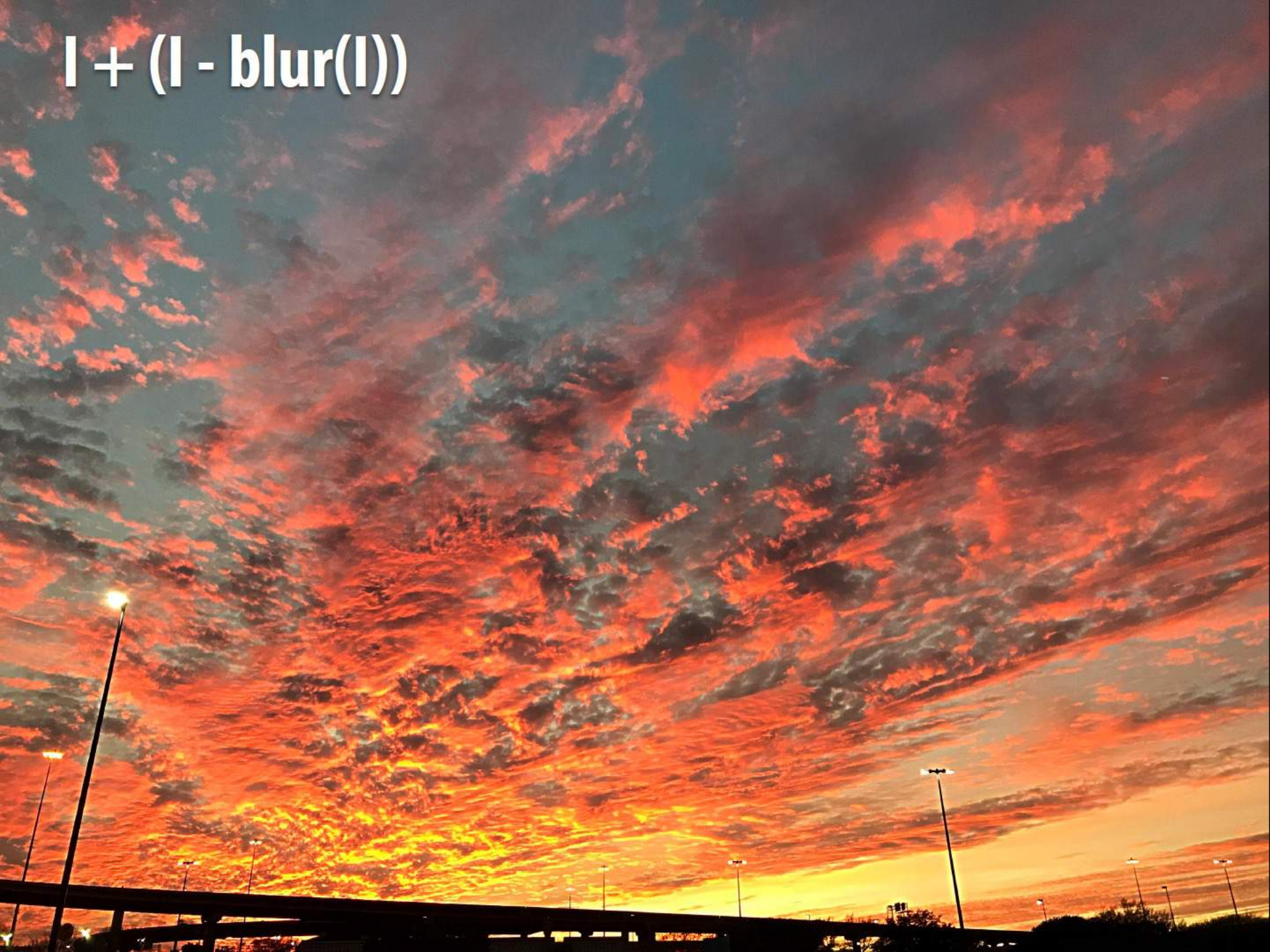
Blur(I)



I - blur(I)



$I + (I - \text{blur}(I))$



What does convolution with these filters do?

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

**Extracts horizontal
gradients**

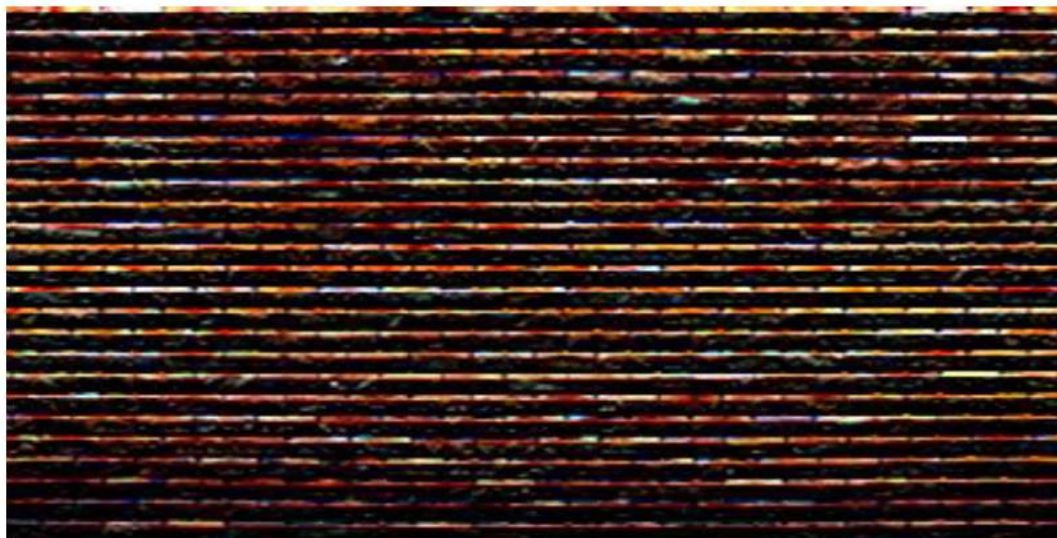
$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

**Extracts vertical
gradients**

Gradient detection filters



Horizontal gradients



Vertical gradients

Note: you can think of a filter as a “detector” of a pattern, and the magnitude of a pixel in the output image as the “response” of the filter to the region surrounding each pixel in the input image (this is a common interpretation in computer vision)

Bilateral filter

Original

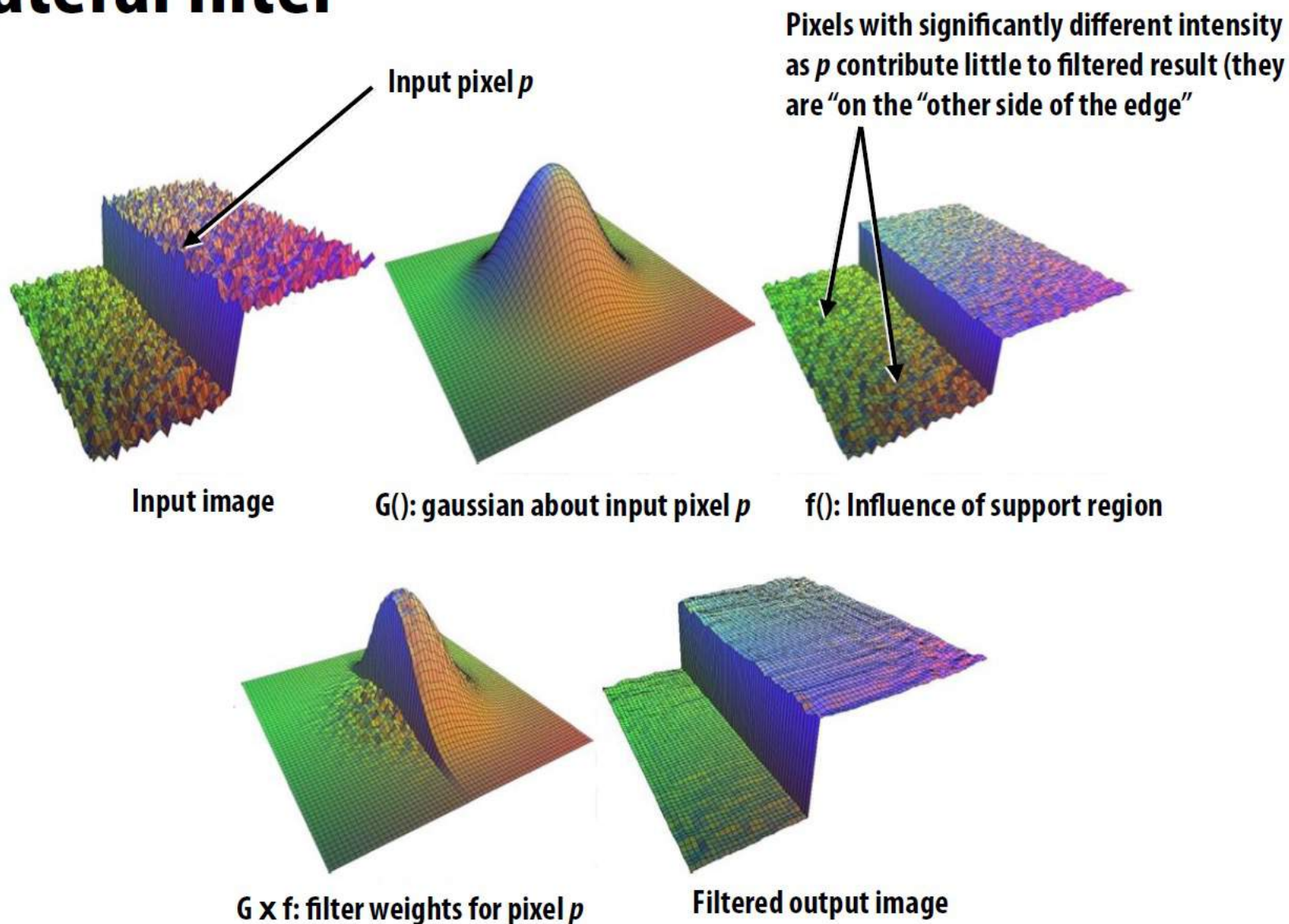


After bilateral filter

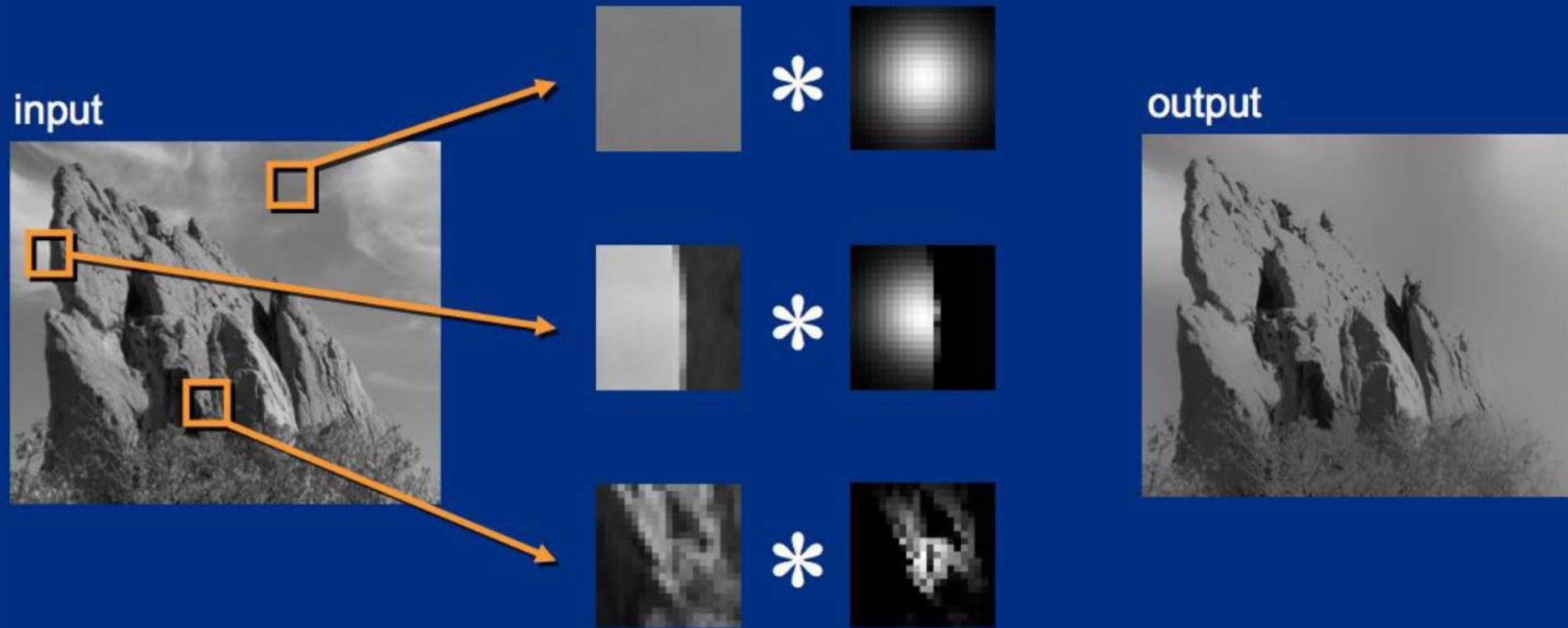


Example use of bilateral filter: removing noise while preserving image edges

Bilateral filter



Bilateral filter: kernel depends on image content

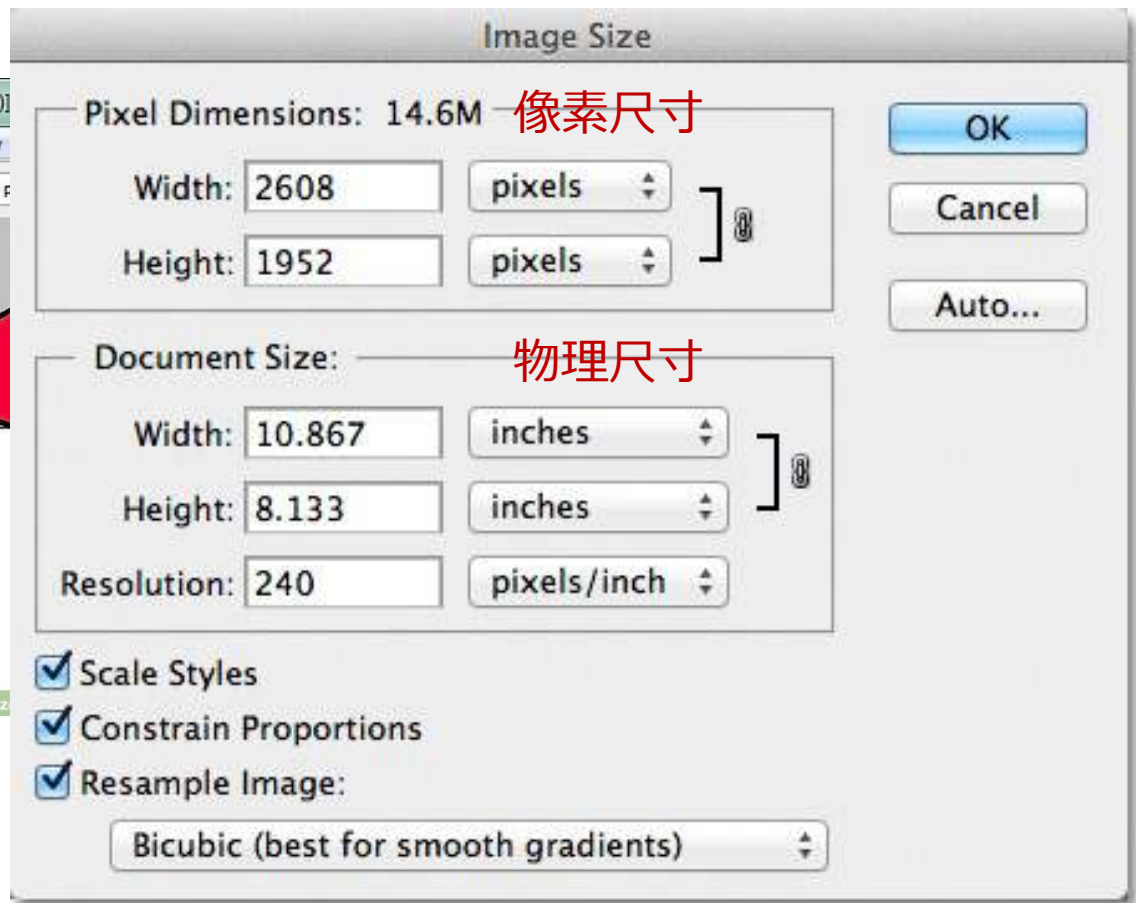
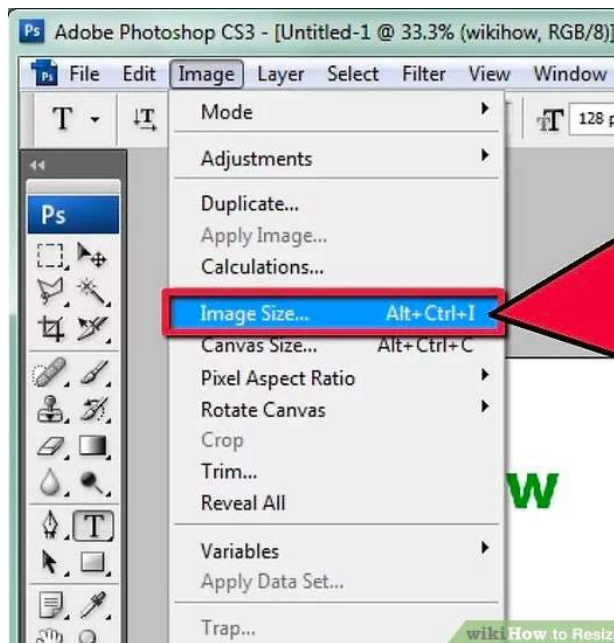


Today

- Image processing basics.
- **Image sampling.**
- Image magnification.

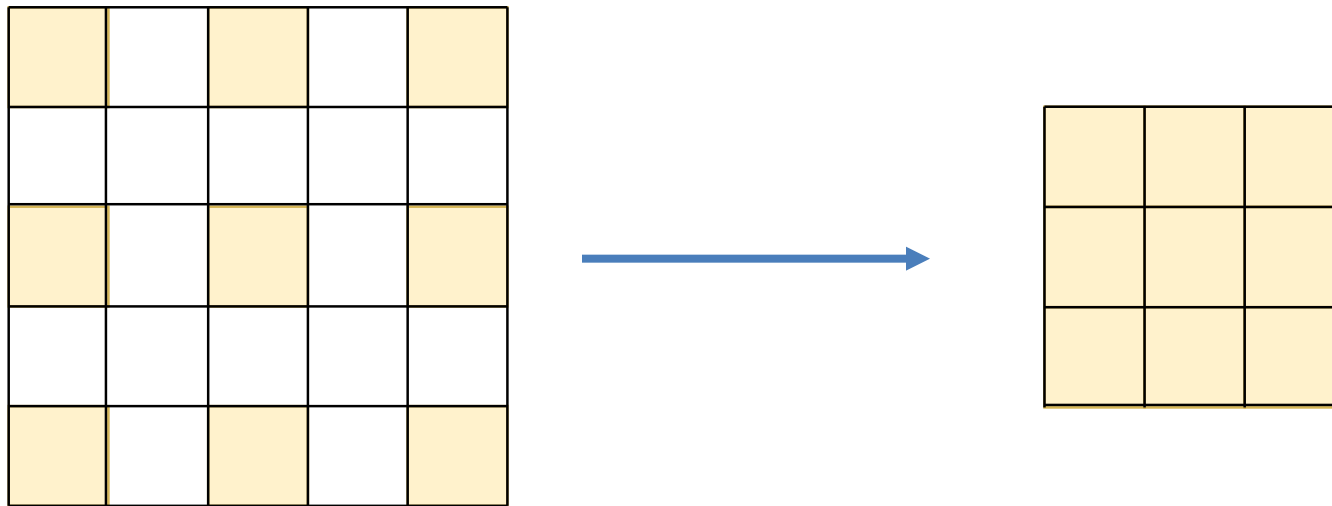
Image resizing

Change image size / resolution in Photoshop



Sampling

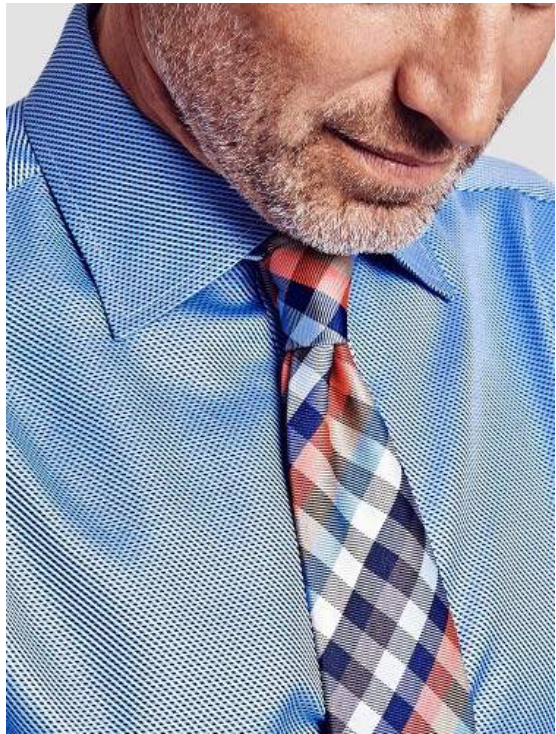
Reducing image size – down-sampling



Is sampling really so easy?

Moiré Patterns in Imaging

[mwa:]



lystit.com

Skip odd rows and columns

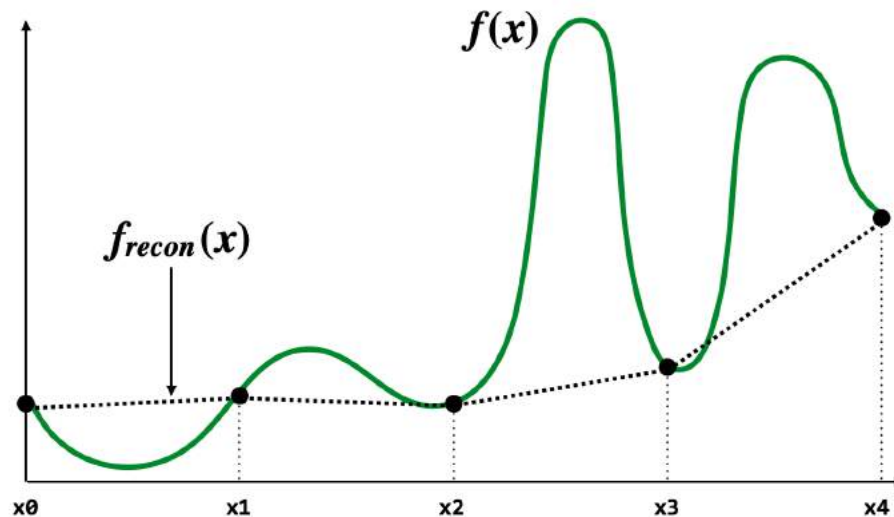
Is sampling really so easy?

Wagon Wheel Illusion (False Motion)



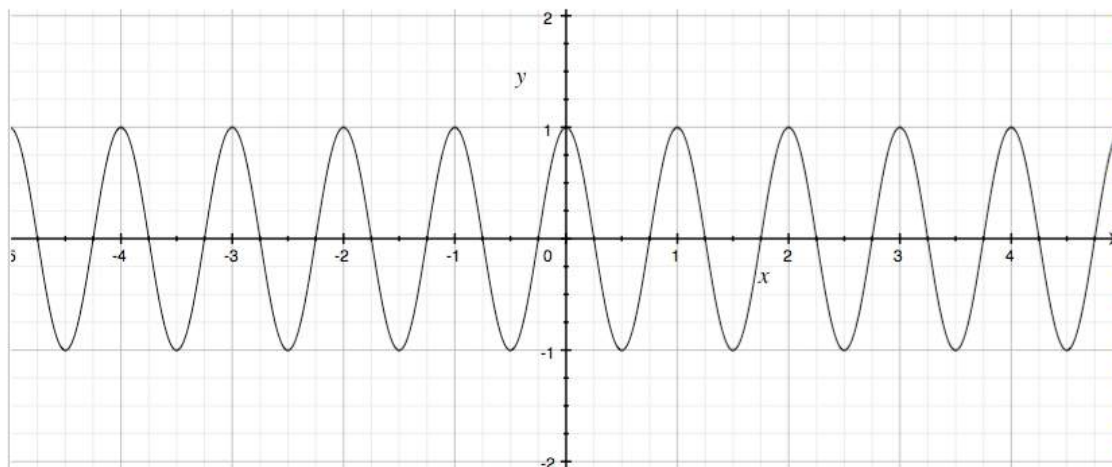
Aliasing

- Aliasing - artifacts due to sampling
- Why does aliasing happen?
 - Signals are **changing too fast** but **sampled too slow**

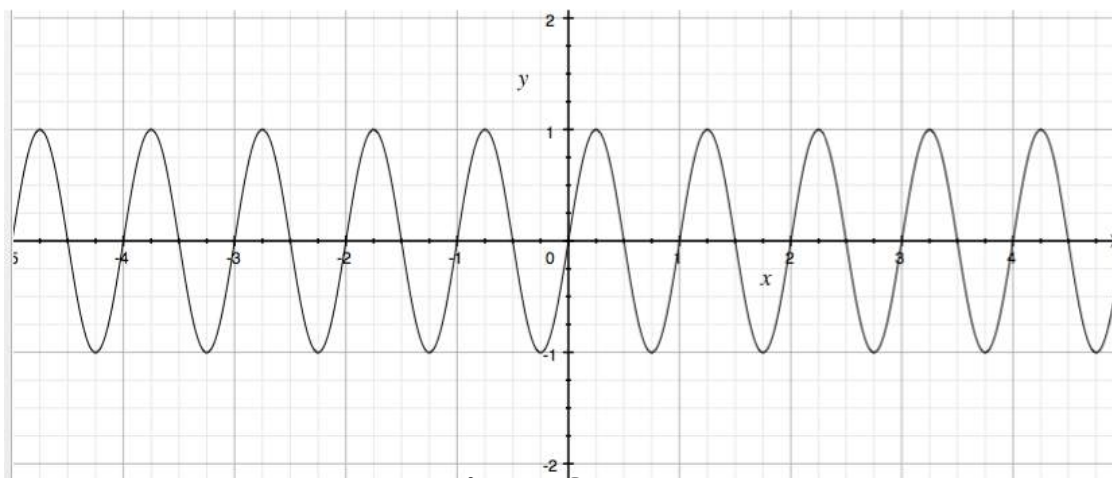


How to mathematically describe the
changing speed of a signal?

Sines and Cosines



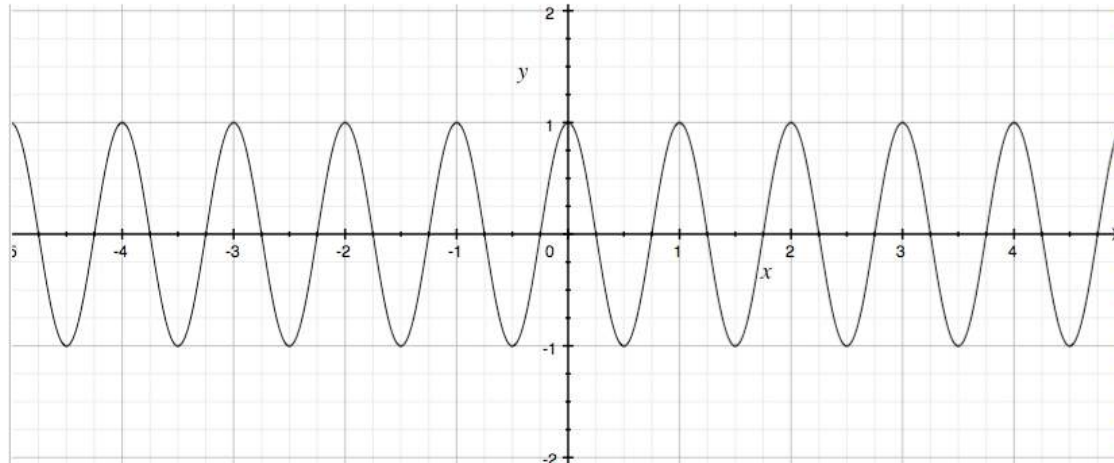
$$\cos 2\pi x$$



$$\sin 2\pi x$$

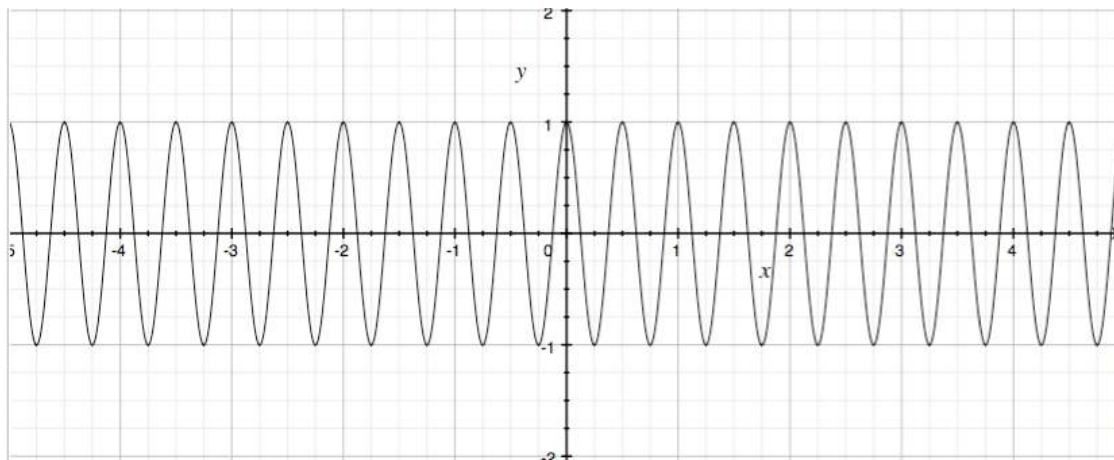
Frequencies $\cos 2\pi f x$

$$f = \frac{1}{T}$$



$$\cos 2\pi x$$

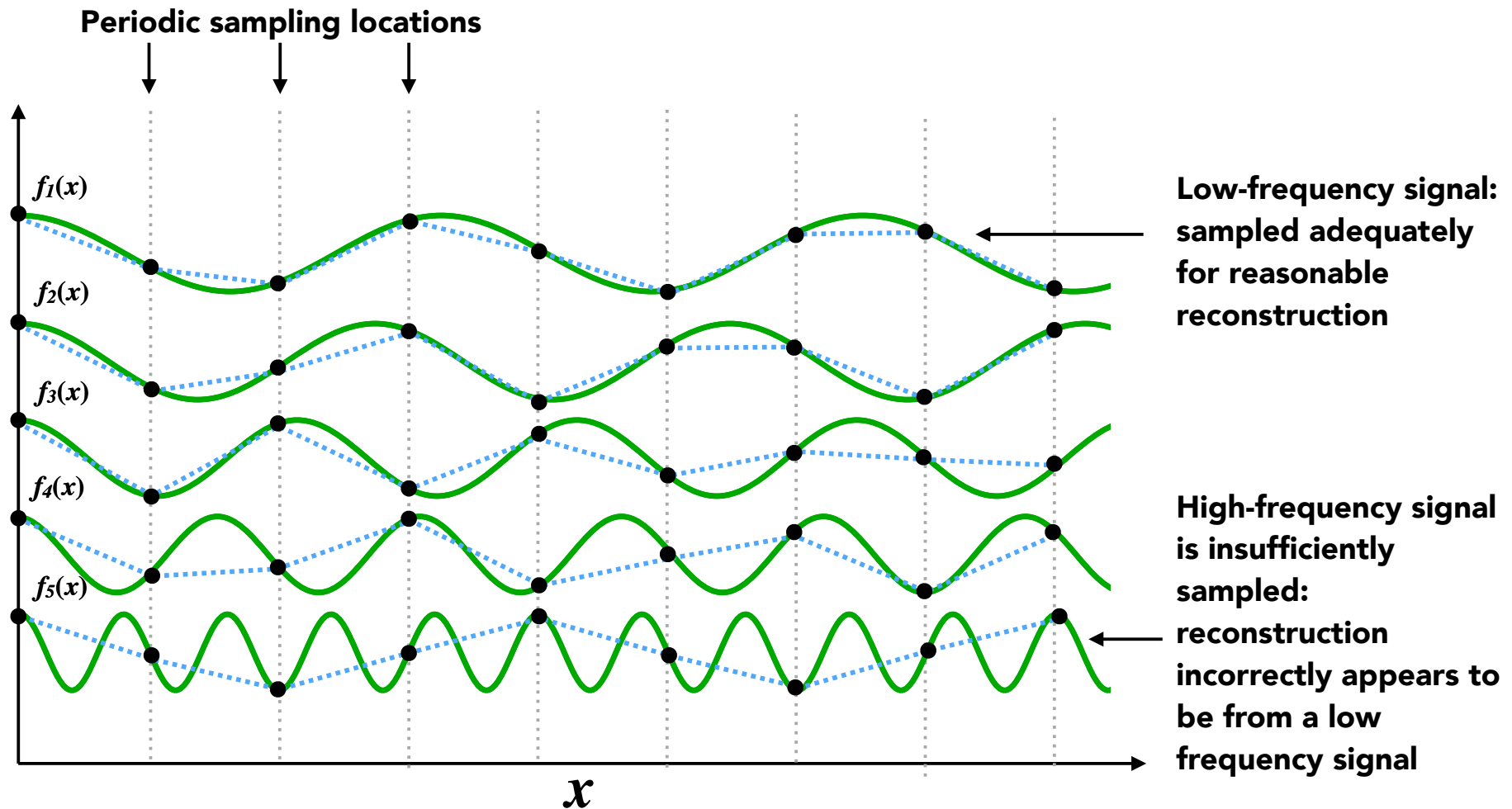
$$f = 1$$



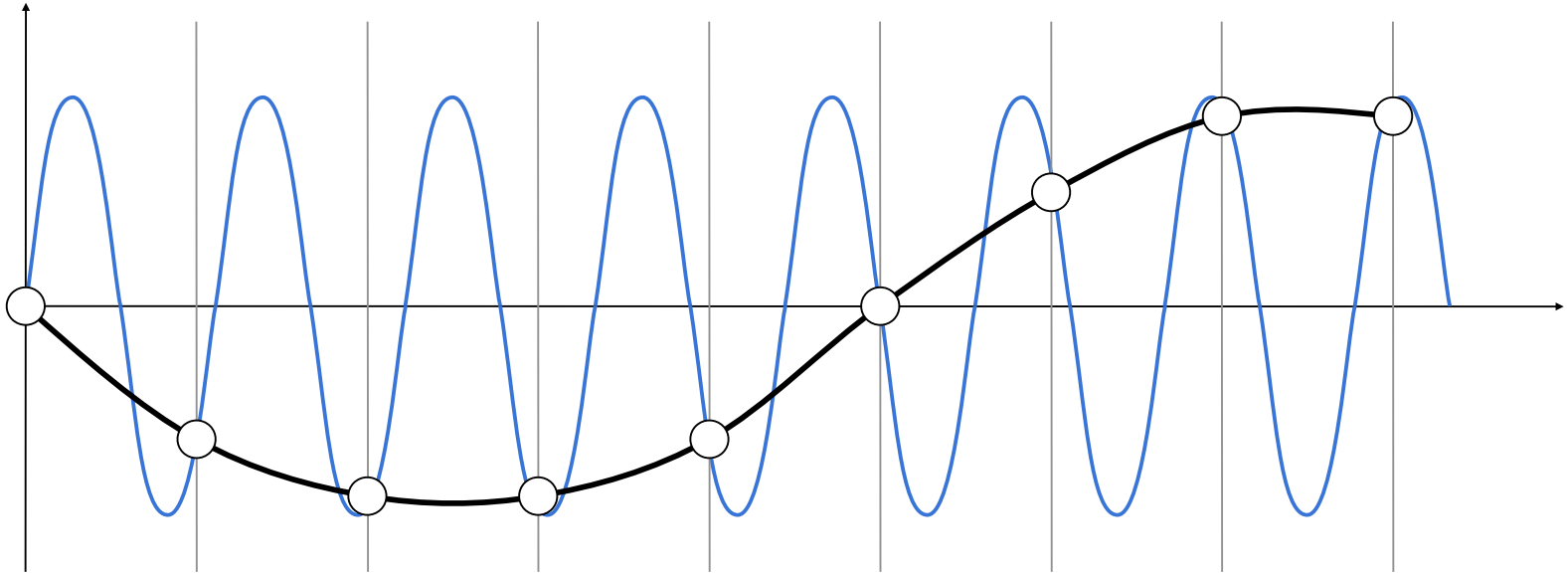
$$\cos 4\pi x$$

$$f = 2$$

Higher Frequencies Need Faster Sampling



Undersampling Creates Frequency Aliases



High-frequency signal is insufficiently sampled: samples erroneously appear to be from a low-frequency signal

Two frequencies that are indistinguishable at a given sampling rate are called "aliases"

What are the frequencies of arbitrary signals

Fourier, Joseph (1768-1830)



French mathematician who discovered that any periodic motion can be written as a superposition of sinusoidal and cosinusoidal vibrations. He developed a mathematical theory of [heat](#) 🌡️ in *Théorie Analytique de la Chaleur* (*Analytic Theory of Heat*), (1822), discussing it in terms of differential equations.

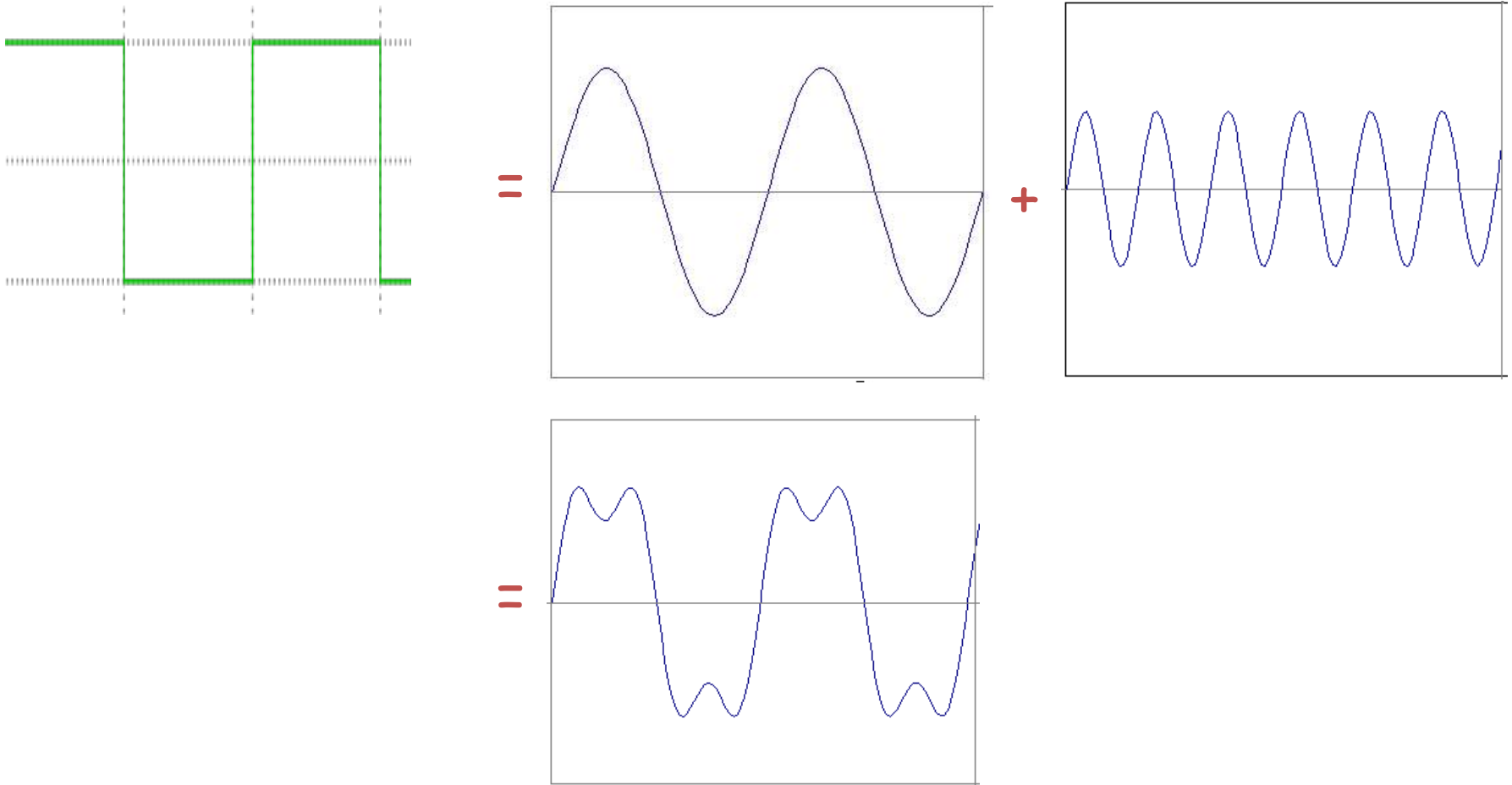
Fourier was a friend and advisor of Napoleon. [Fourier believed that his health would be improved by wrapping himself up in blankets, and in this state he tripped down the stairs in his house and killed himself.](#) The paper of [Galois](#) which he had taken home to read shortly before his death was never recovered.

SEE ALSO: [Galois](#)

Additional biographies: [MacTutor](#) (St. Andrews), [Bonn](#)

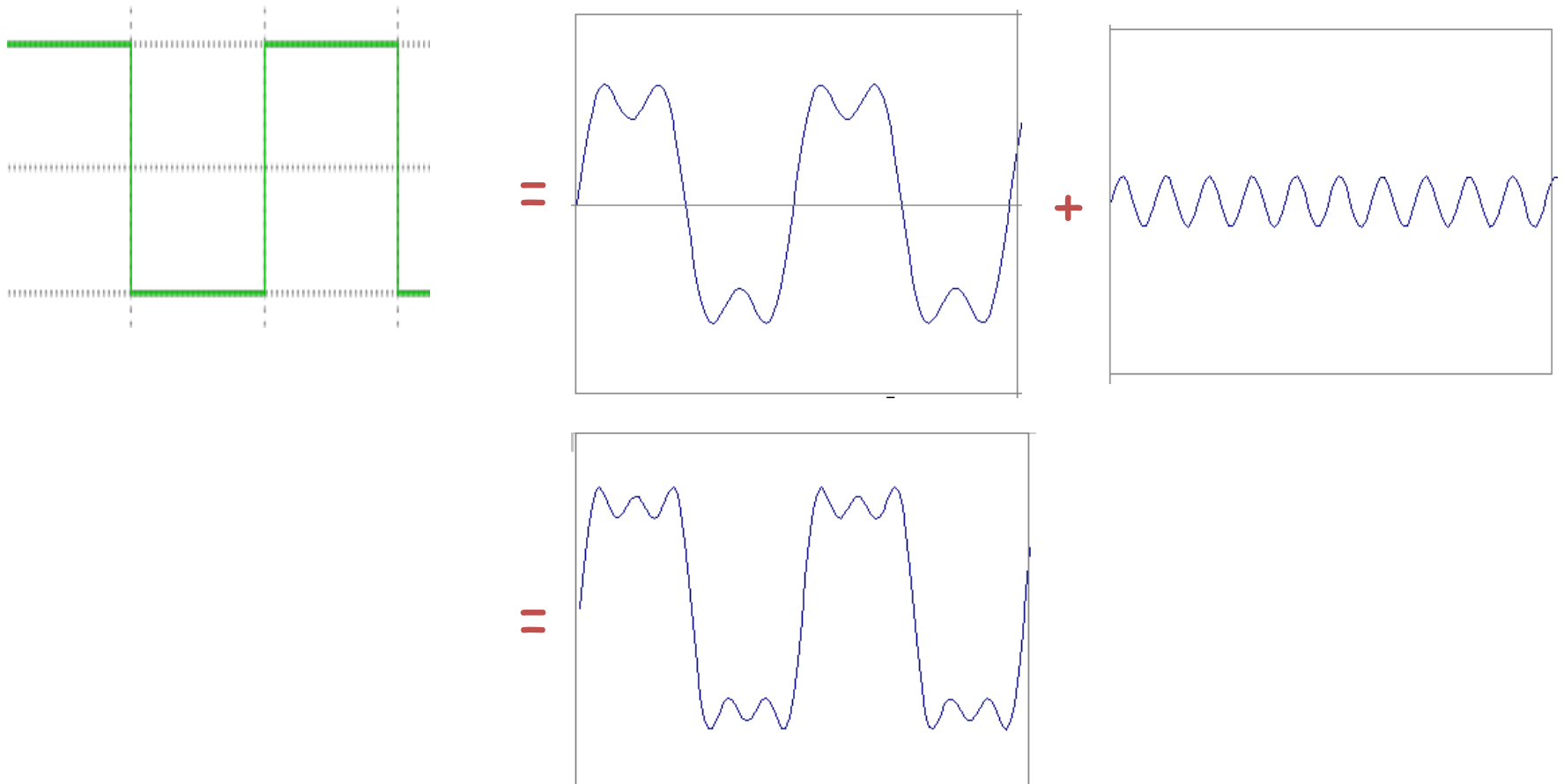
Fourier Transform

- Represent a function as a weighted sum of sines and cosines.



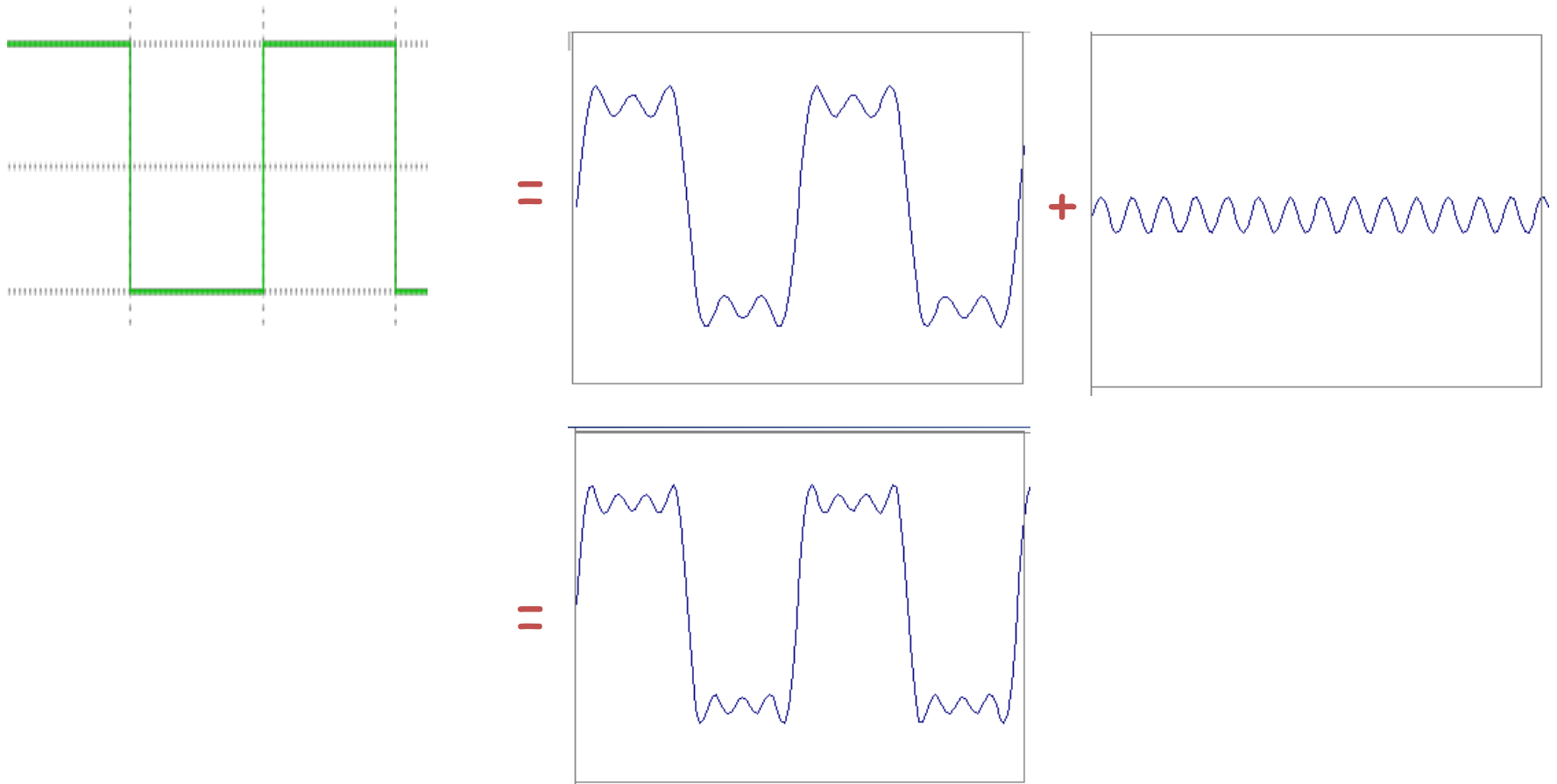
Fourier Transform

- Represent a function as a weighted sum of sines and cosines.



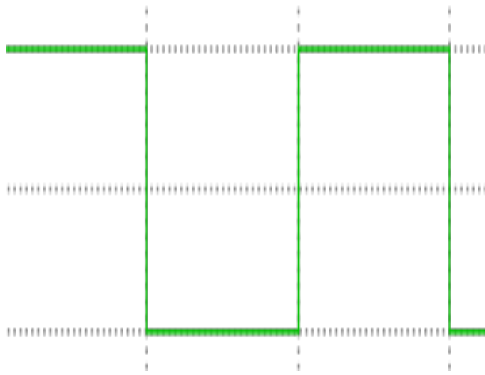
Fourier Transform

- Represent a function as a weighted sum of sines and cosines.

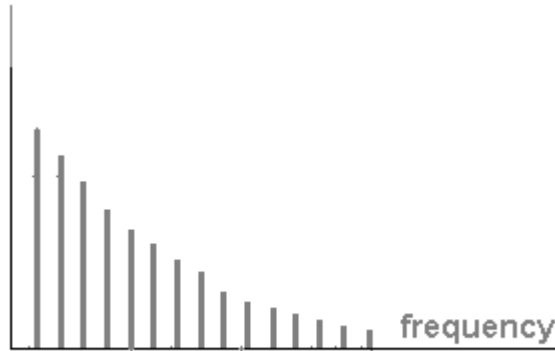


Fourier Transform

- Represent a function as a weighted sum of sines and cosines.

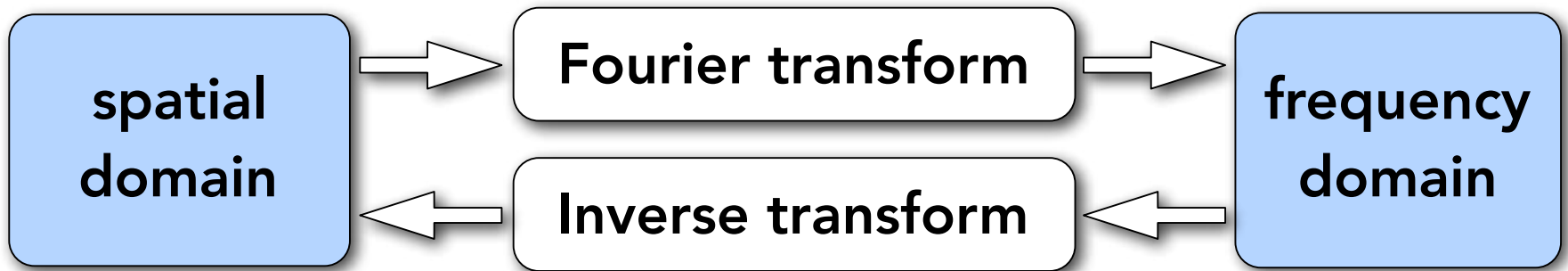


$$= \frac{A}{2} + \frac{2A \cos(t\omega)}{\pi} - \frac{2A \cos(3t\omega)}{3\pi} + \frac{2A \cos(5t\omega)}{5\pi} - \frac{2A \cos(7t\omega)}{7\pi} + \dots$$



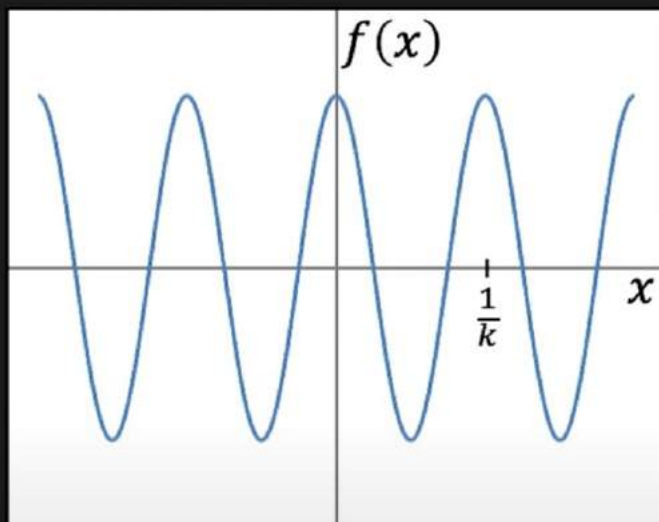
Fourier Transform

- $F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi ux} dx$
- $f(x) = \int_{-\infty}^{\infty} F(u) e^{i2\pi ux} du$
- x : space, u : frequency, $e^{i\theta} = \cos\theta + i\sin\theta$,
 $i = \sqrt{-1}$



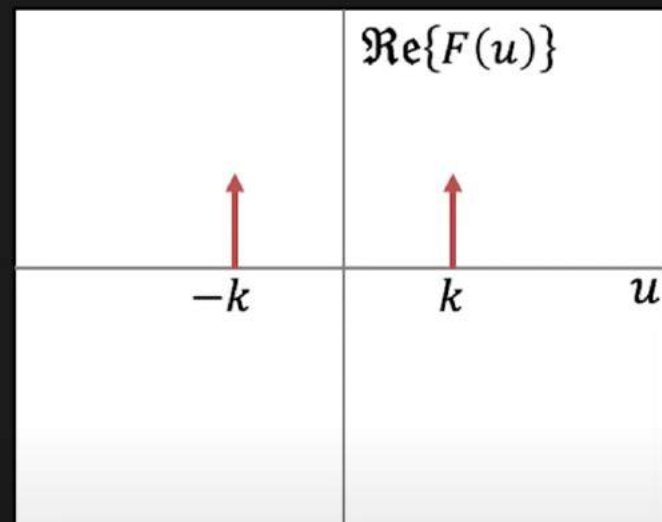
Fourier Transform of sinusoids

Signal $f(x)$



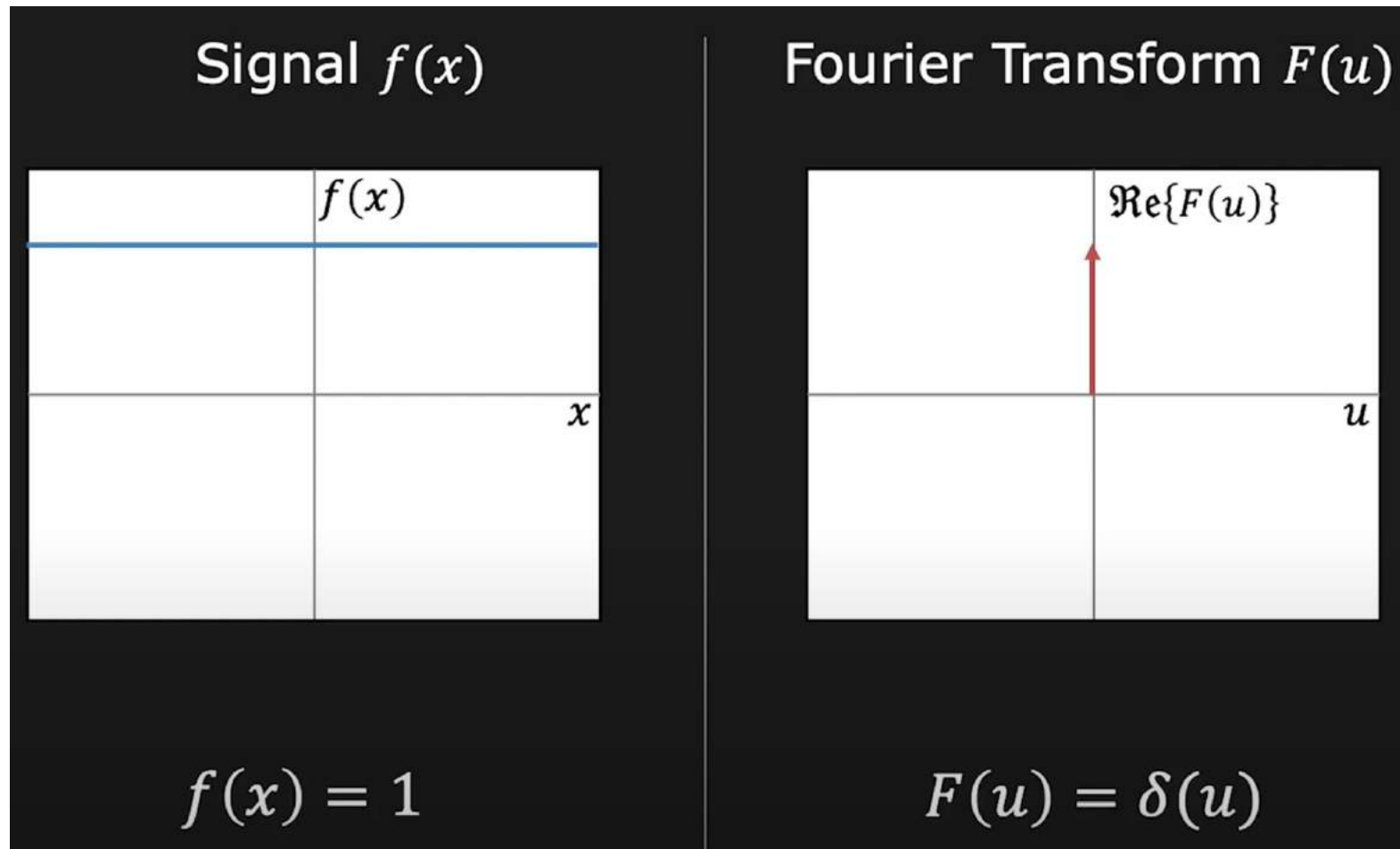
$$f(x) = \cos 2\pi kx$$

Fourier Transform $F(u)$

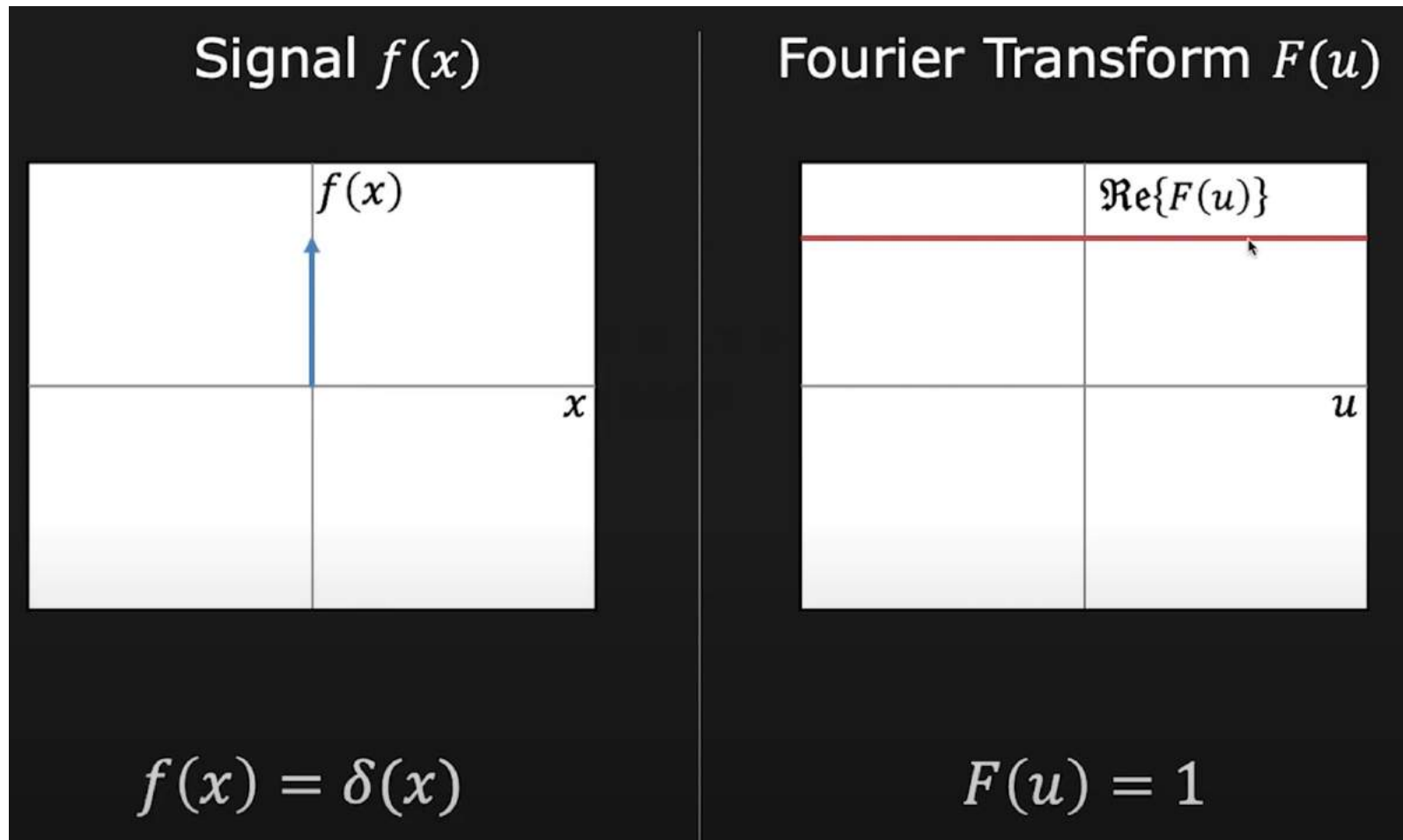


$$F(u) = \frac{1}{2}[\delta(u + k) + \delta(u - k)]$$

Fourier Transform of constant function

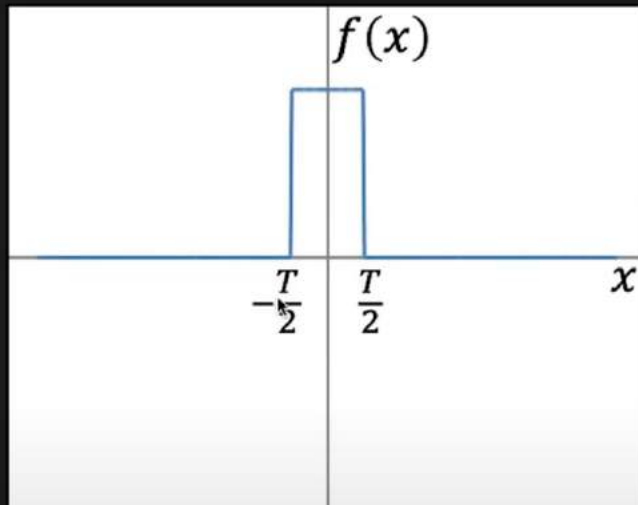


Fourier Transform of Dirac function



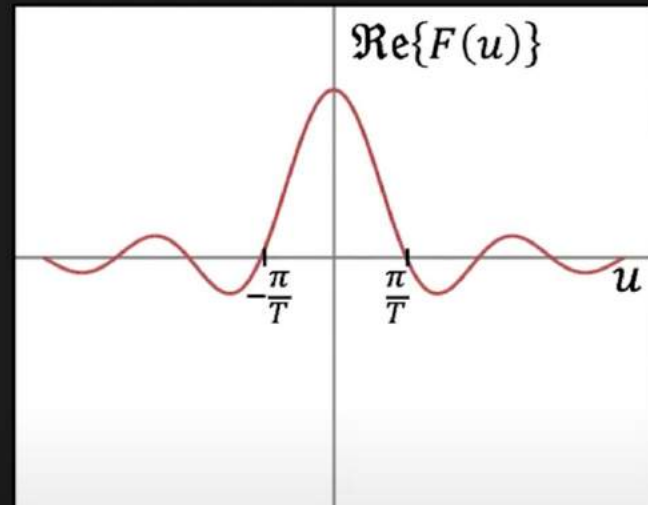
Fourier Transform of box function

Signal $f(x)$



$$f(x) = \text{Rect}\left(\frac{x}{T}\right)$$

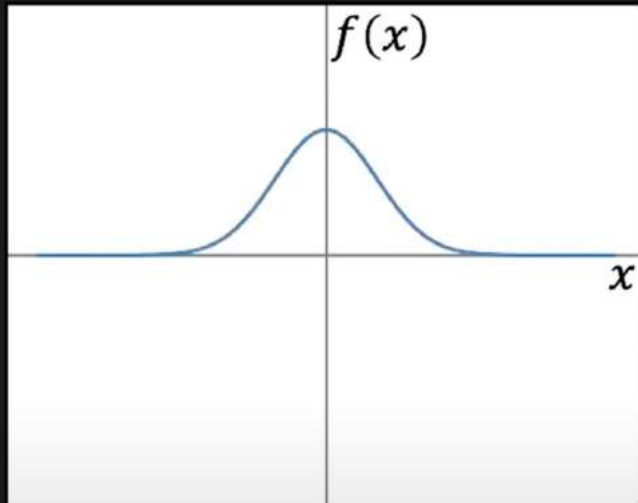
Fourier Transform $F(u)$



$$F(u) = T \text{sinc } Tu$$

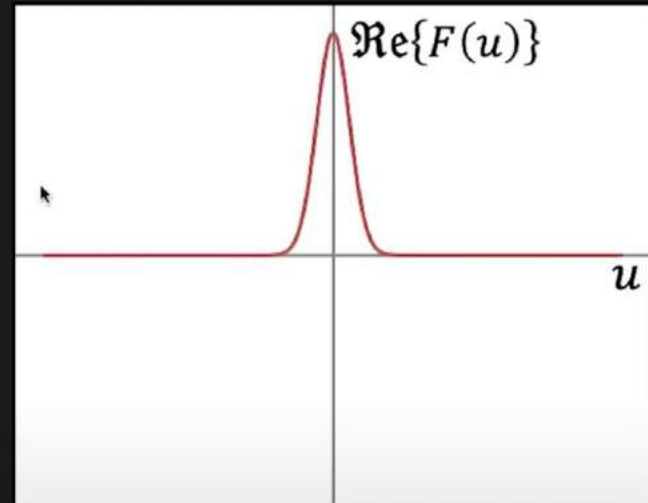
Fourier Transform of Gaussian function

Signal $f(x)$



$$f(x) = e^{-ax^2}$$

Fourier Transform $F(u)$

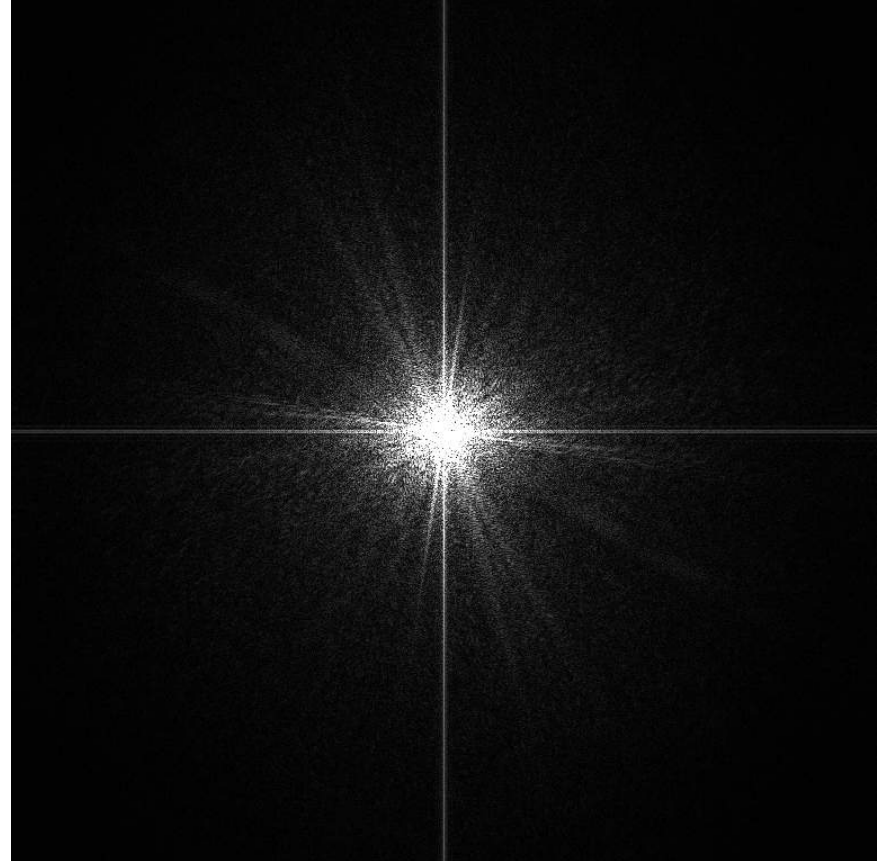


$$F(u) = \sqrt{\pi/a} e^{-\pi^2 u^2 / a}$$

Convolution Theorem

Spatial Domain		Frequency Domain
$g(x) = f(x) * h(x)$ Convolution	$\longleftrightarrow$	$G(u) = F(u) H(u)$ Multiplication
$g(x) = f(x) h(x)$ Multiplication	$\longleftrightarrow$	$G(u) = F(u) * H(u)$ Convolution

Visualizing frequency content of images



Convolution Theorem in 2D

Spatial
Domain



*

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

=



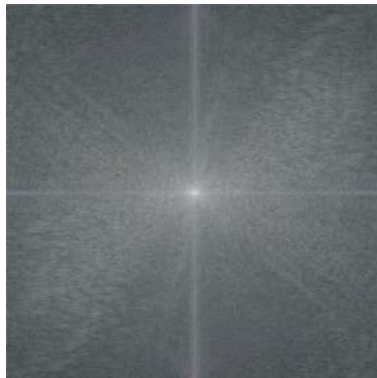
Fourier
Transform



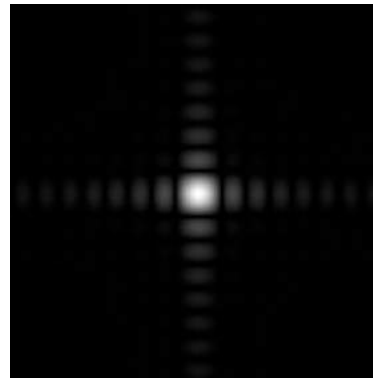
Inv. Fourier
Transform



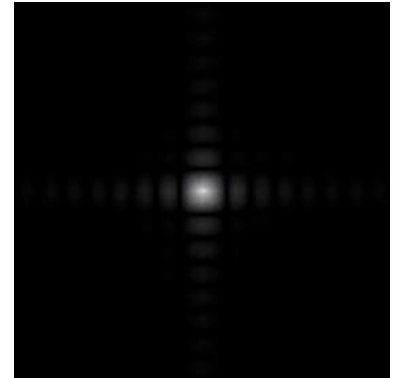
Frequency
Domain



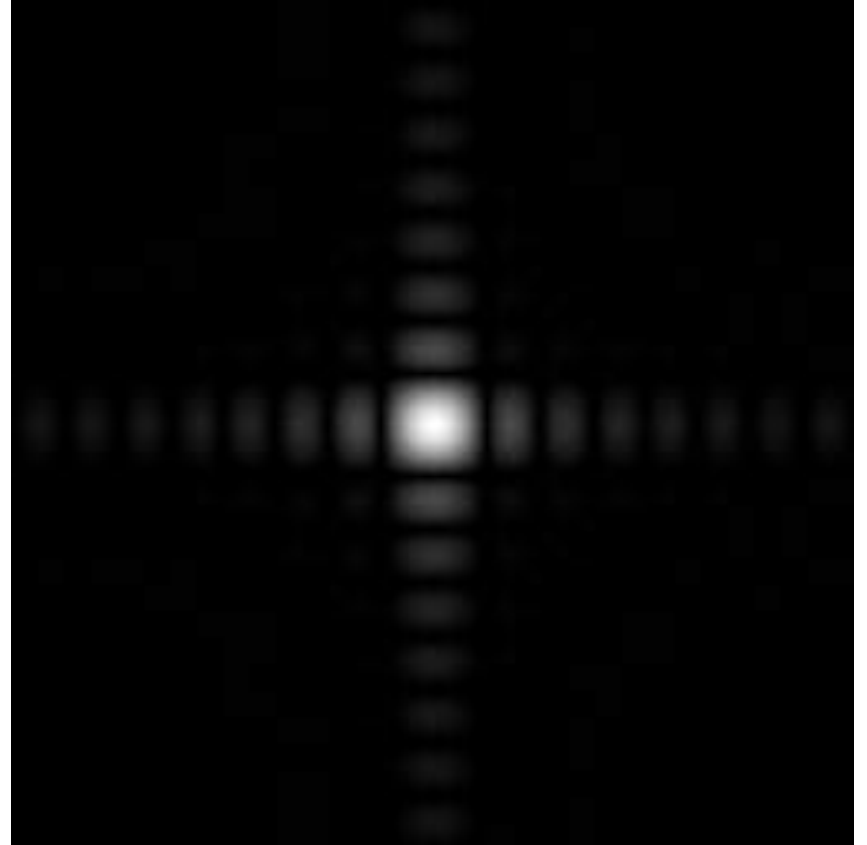
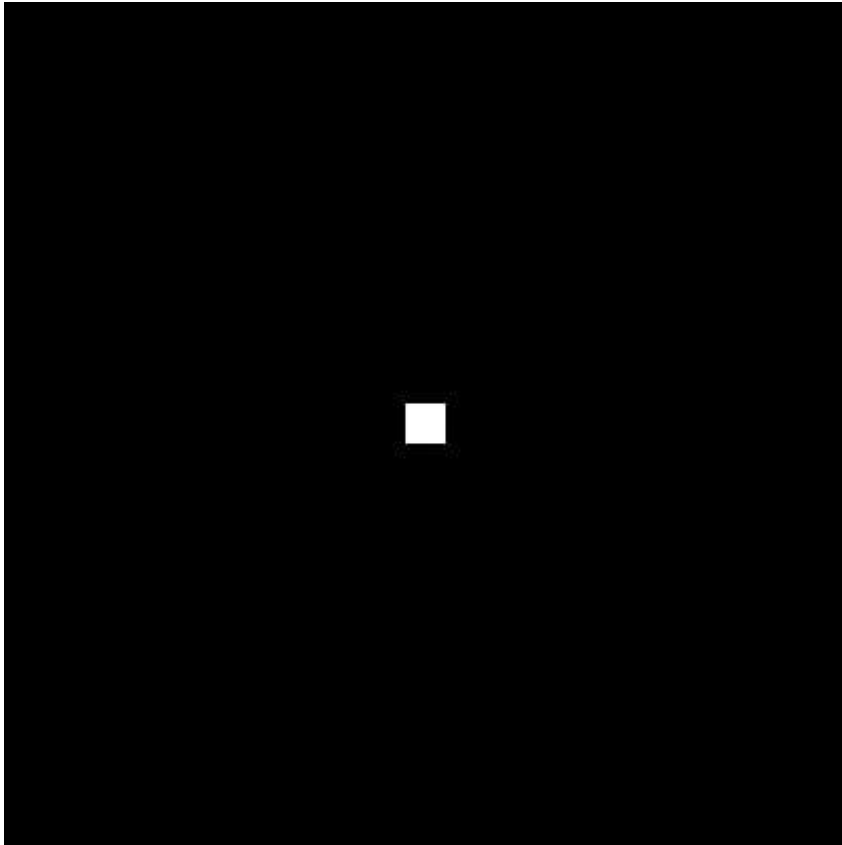
x



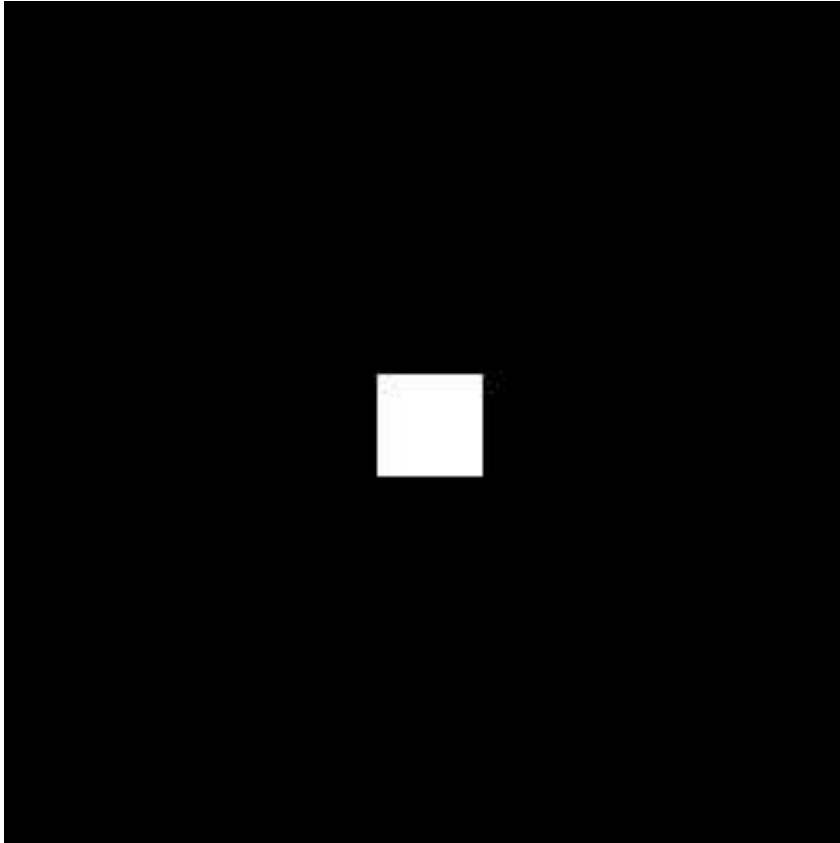
=



Box filter = low-pass filter

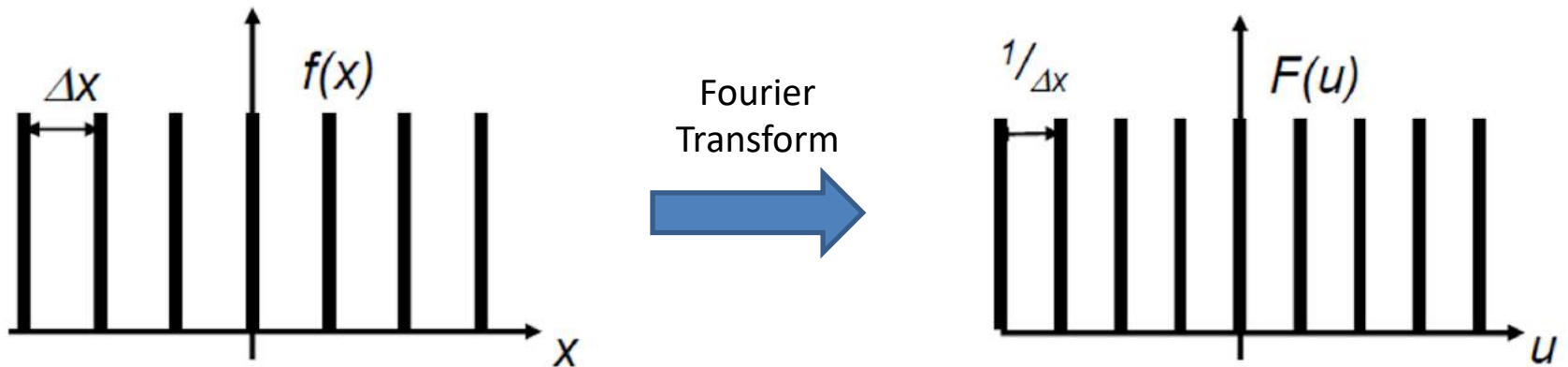


Wider kernel = lower frequency

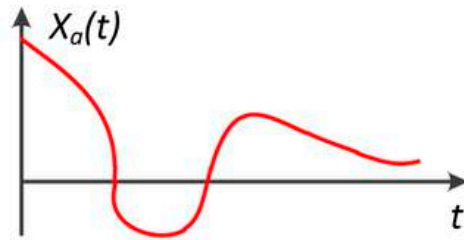


Sampling

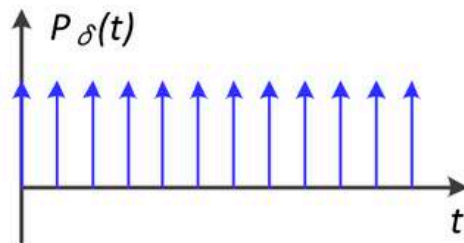
- Sampling a signal = multiply the signal by a Dirac comb function



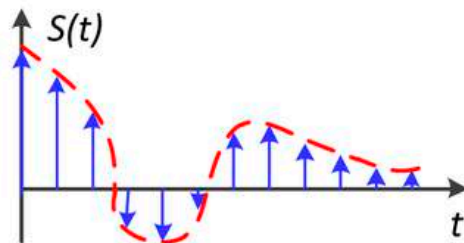
Sampling = Repeating Frequency Contents



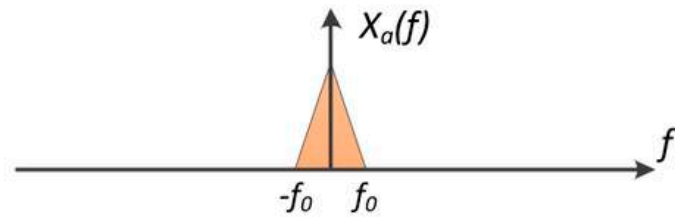
X

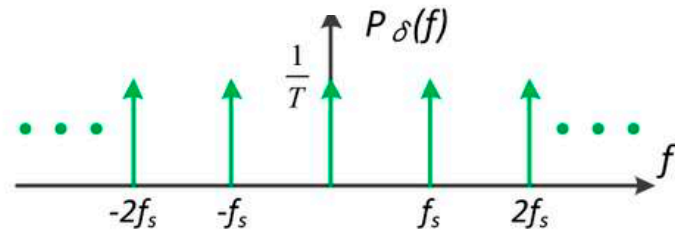


||

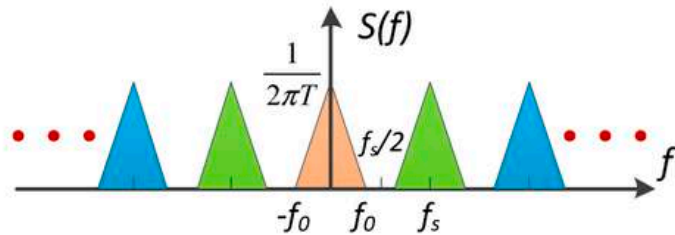


Spatial domain





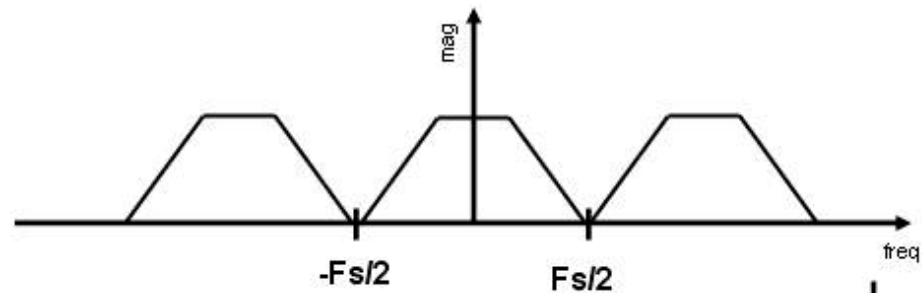
||



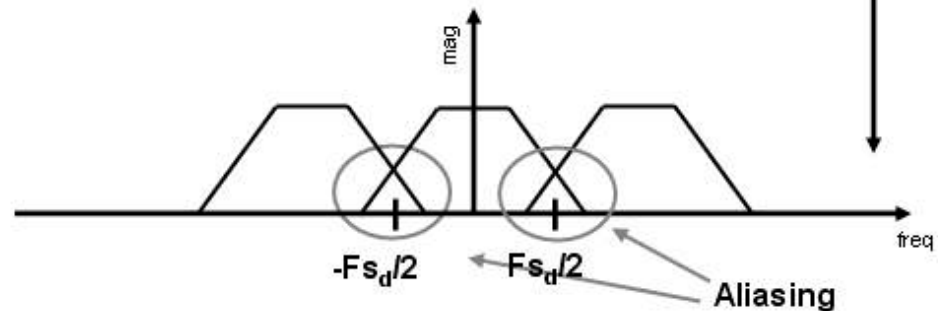
Fourier domain

Aliasing = Mixed Frequency Contents

Dense sampling:



Sparse sampling:



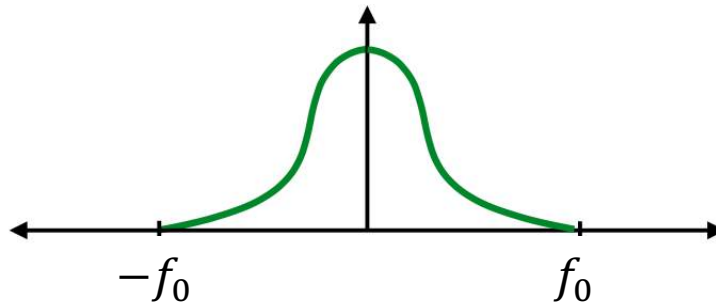
How can we reduce aliasing?

Option 1: Increasing sampling rate

How large is enough?

Nyquist-Shannon theorem

- Consider a band-limited signal: has no frequencies above f_0



- The signal can be perfectly reconstructed if sampled with a frequency larger than $2f_0$

How can we reduce aliasing?

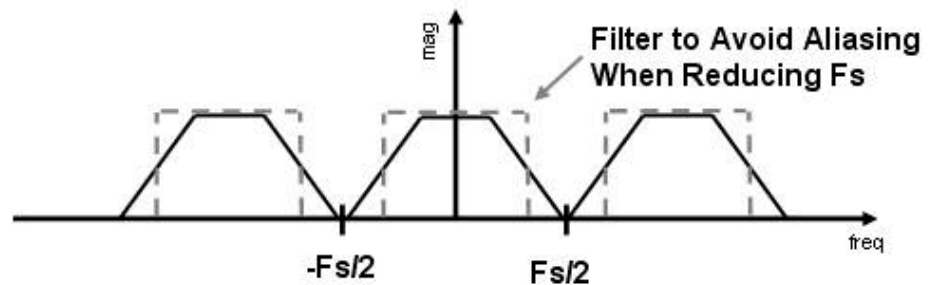
Option 1: Increasing sampling rate

Option 2: Anti-aliasing

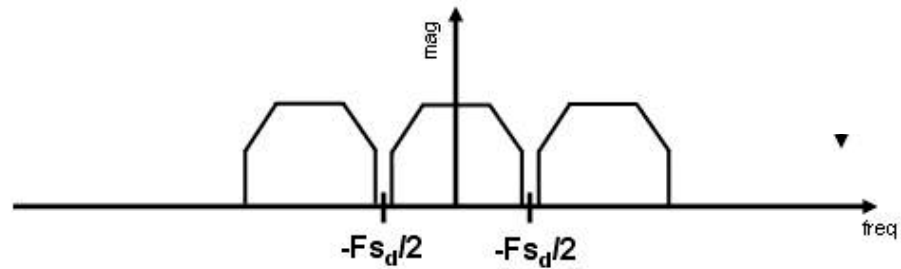
Filtering out high frequencies before sampling

Anti-aliasing = Filtering, then sampling

Filtering



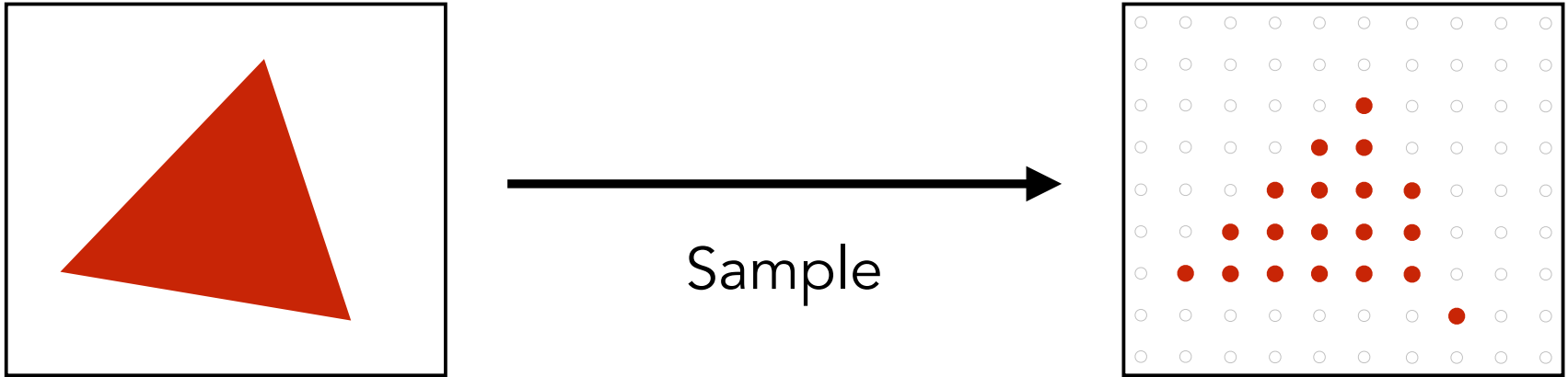
Then sparse sampling



How to filter out high frequencies?

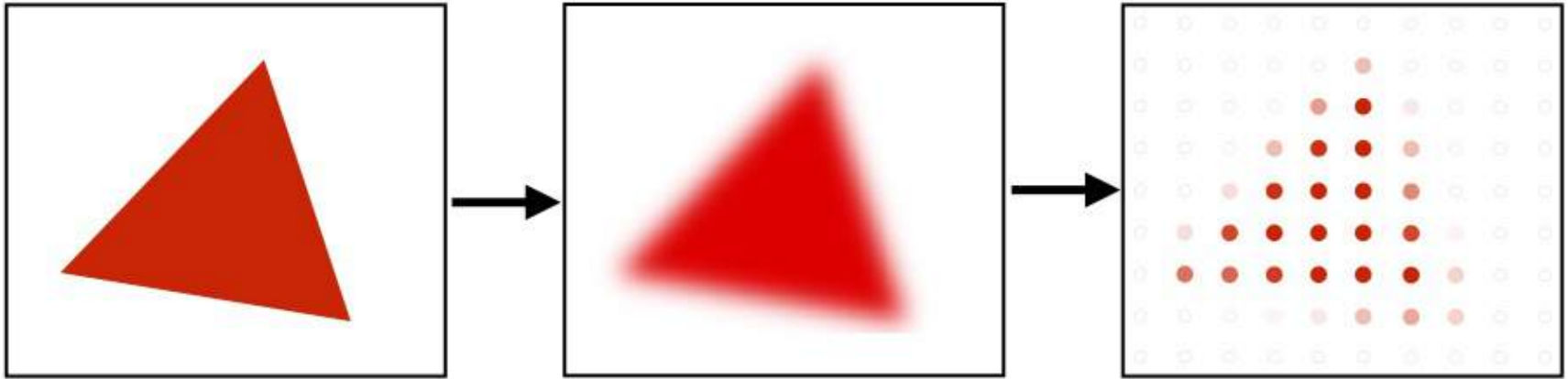
- Filtering = convolution
- Steps for anti-aliasing
 1. Convolve the image with low-pass filters (e.g. Gaussian)
 2. Sample it with a Nyquist rate

Regular Sampling



Note jaggies in rasterized triangle
where pixel values are pure red or white

Antialiased Sampling



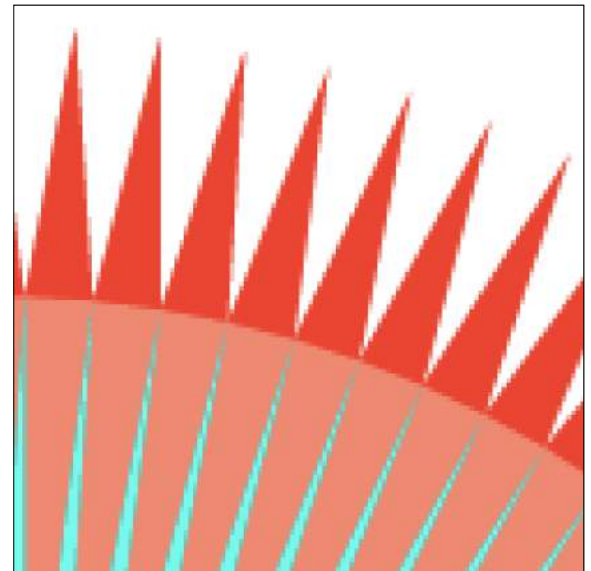
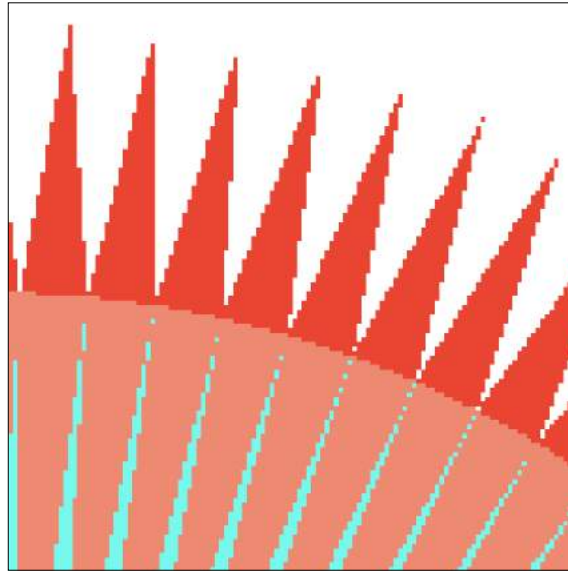
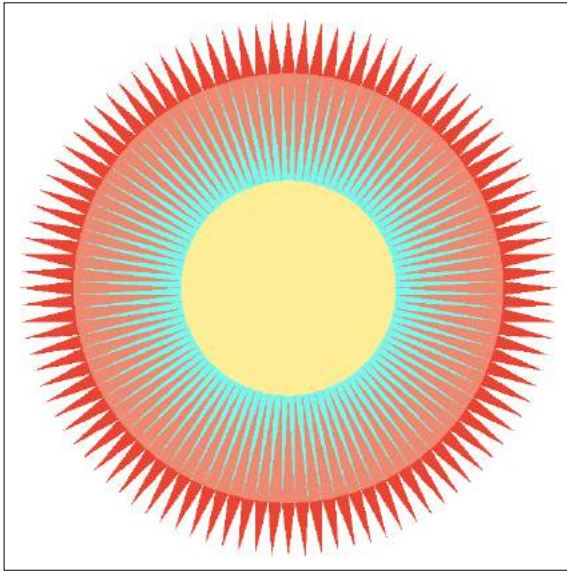
Pre-Filter

(remove frequencies above Nyquist)

Sample

Note antialiased edges in rasterized triangle where pixel values take intermediate values

Antialiasing



Antialiasing



Today

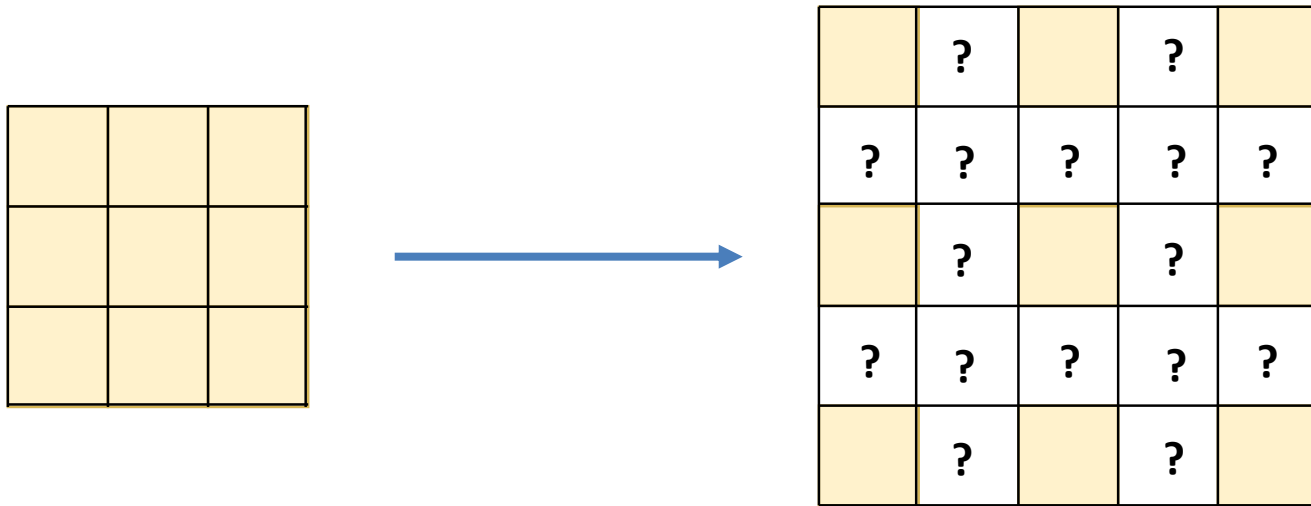
- Image processing basics.
- Image sampling.
- **Image magnification.**

Image magnification

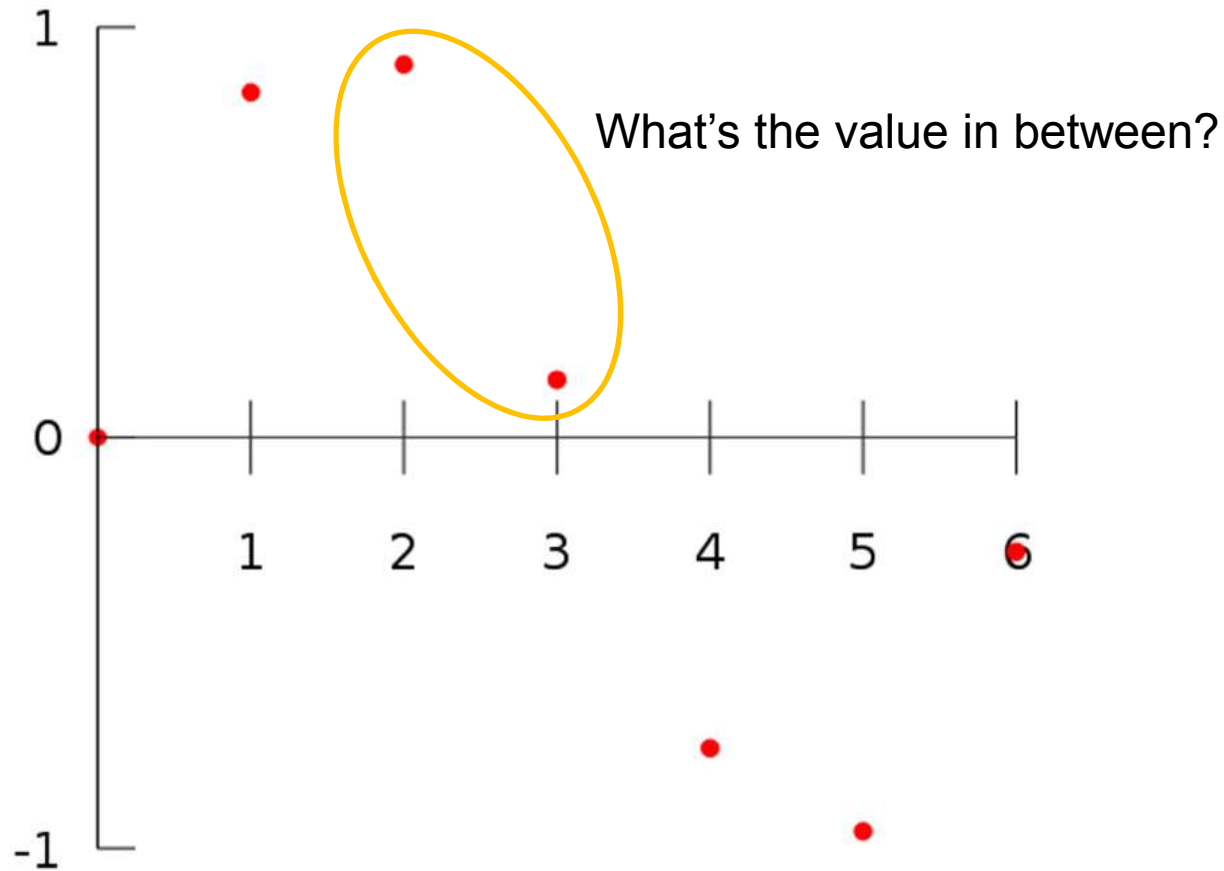


Image magnification

Inverse of down-sampling (up-sampling)



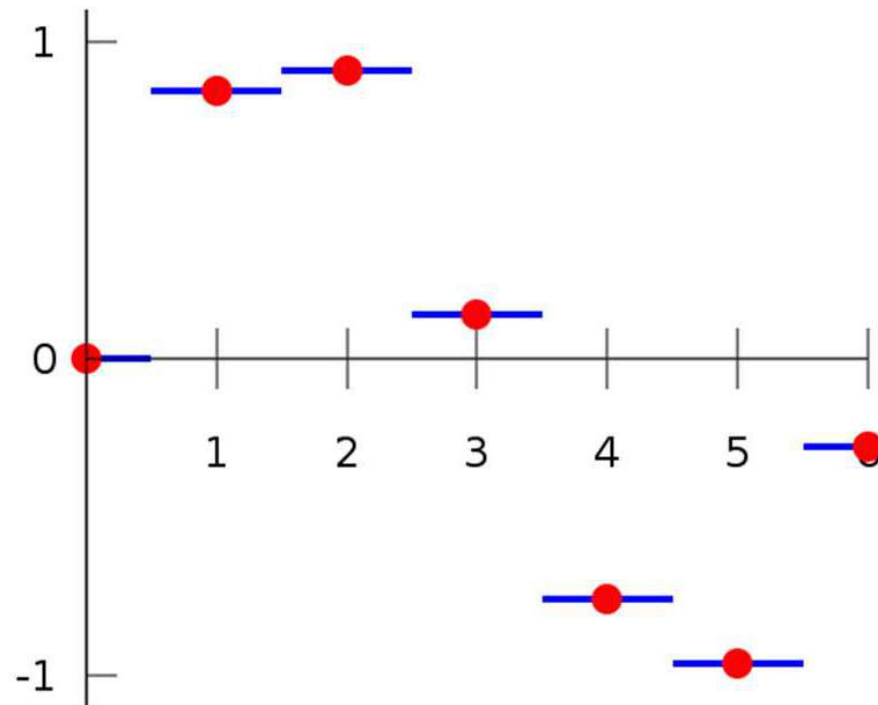
Interpolation



Nearest-neighbor interpolation

Not continuous

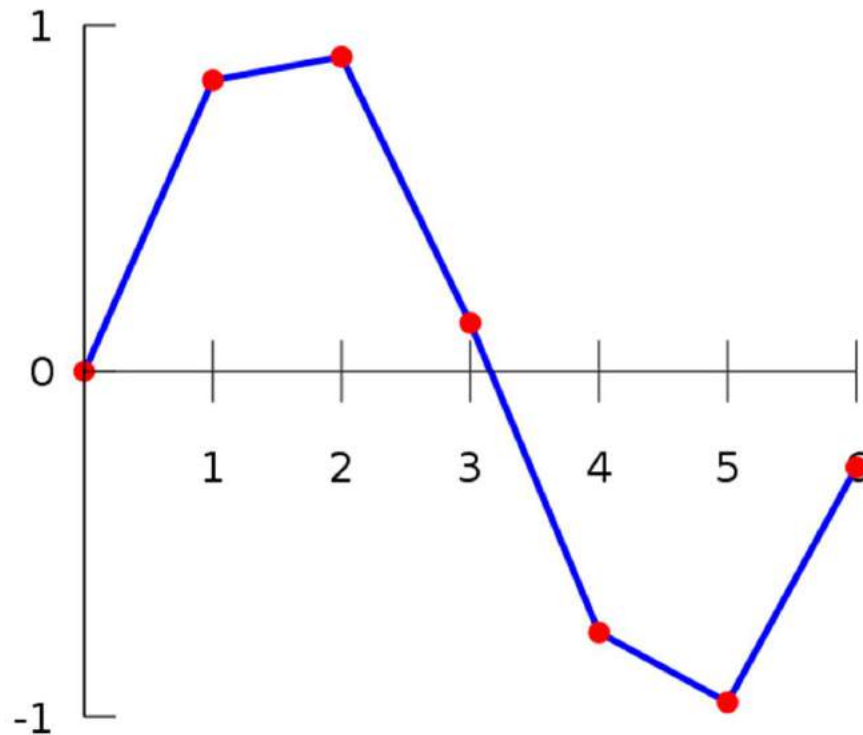
Not smooth



Linear interpolation

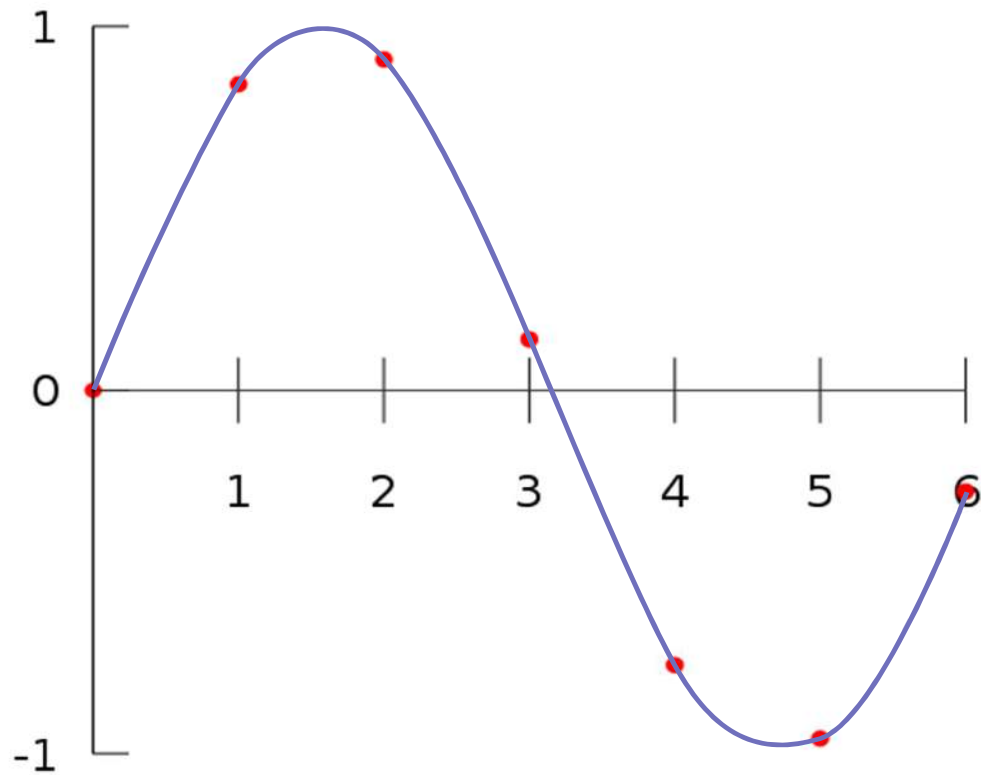
Continuous

Not smooth



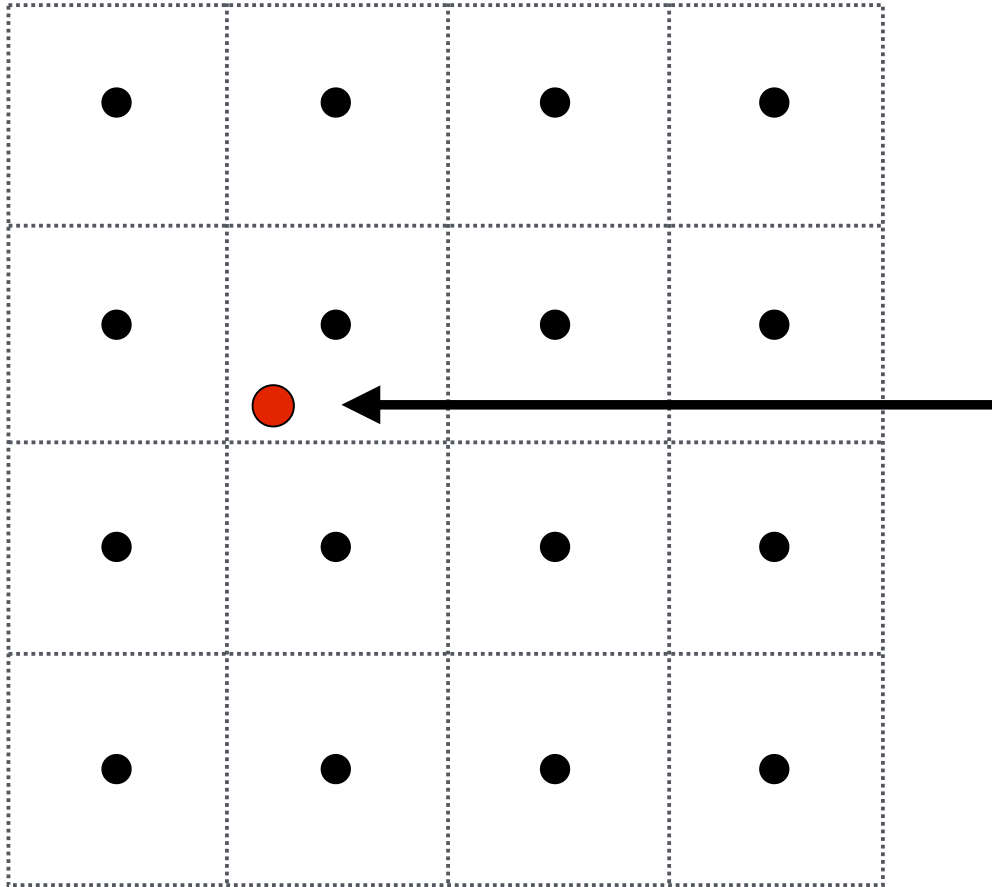
Cubic interpolation

Continuous
Smooth



For each interval: $y = ax^3 + bx^2 + cx + d$

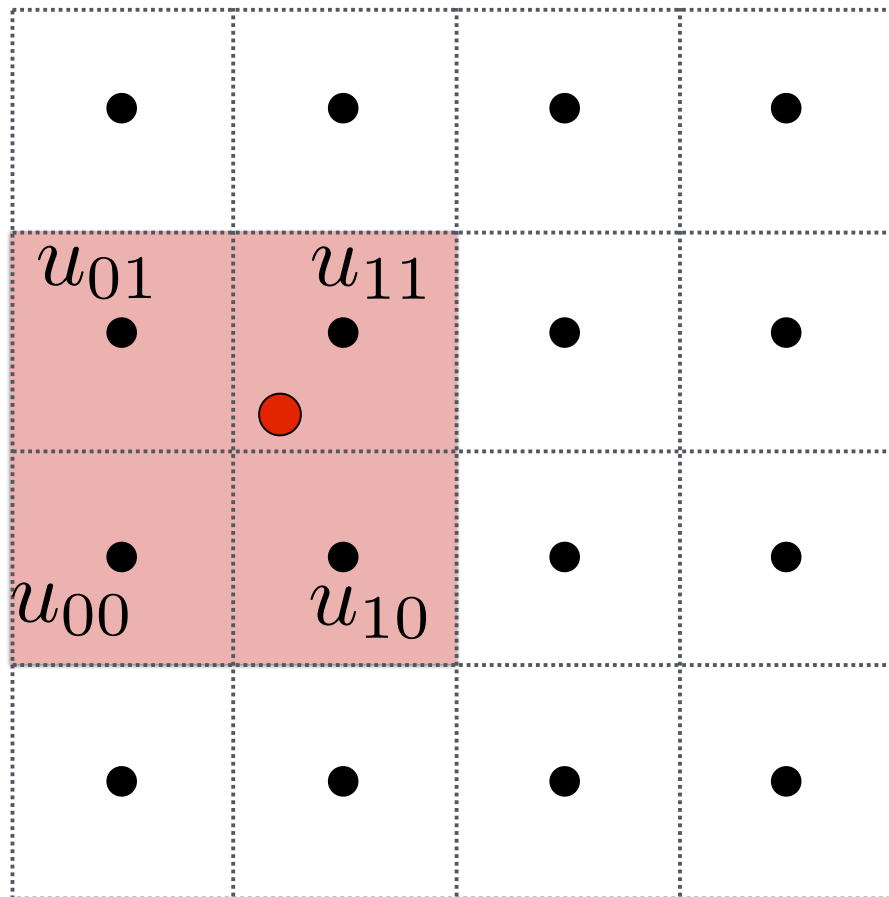
Bilinear Interpolation



Want to sample
texture value $f(x, y)$ at
red point

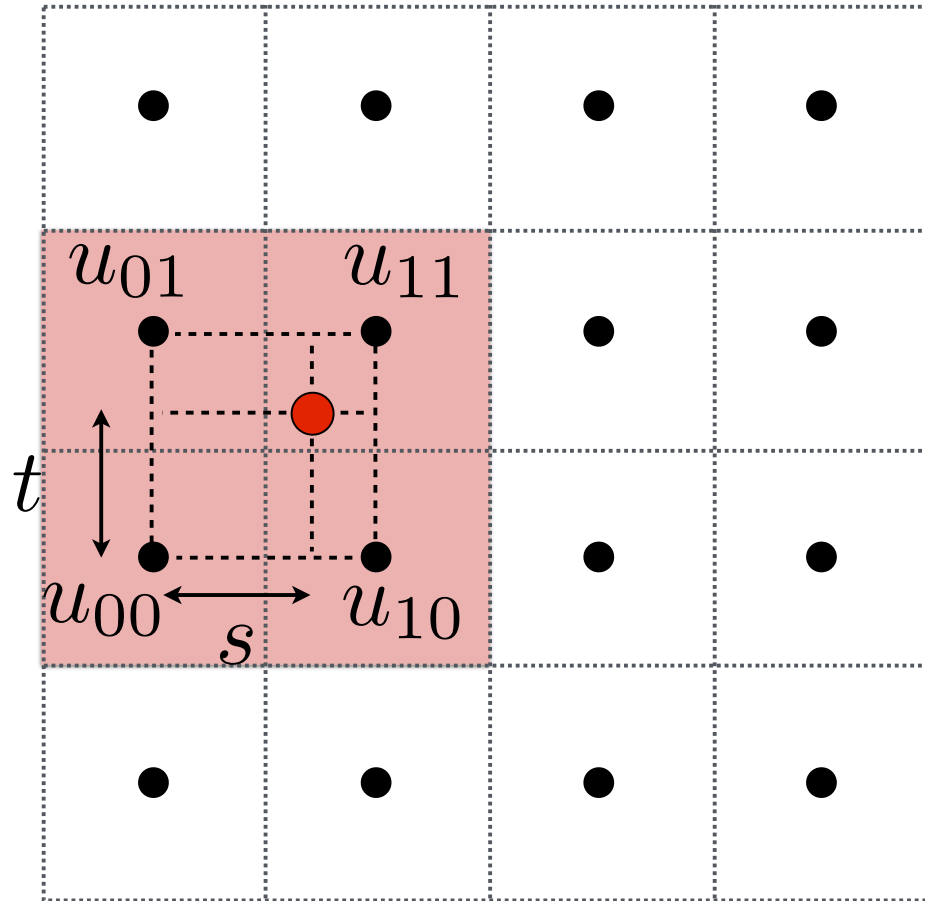
Black points indicate
texture sample
locations

Bilinear Interpolation



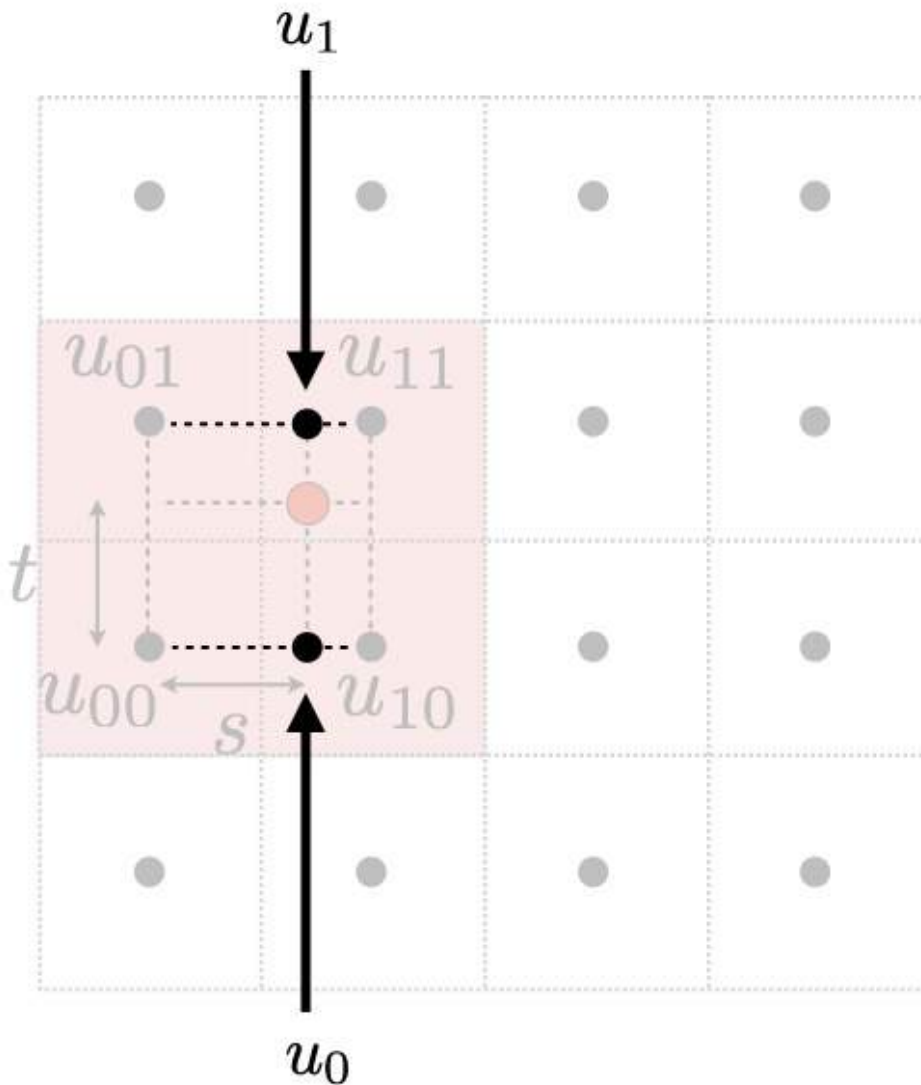
Take 4 nearest sample locations, with texture values as labeled.

Bilinear Interpolation



**And fractional offsets,
(s,t) as shown**

Bilinear Interpolation



Linear interpolation (1D)

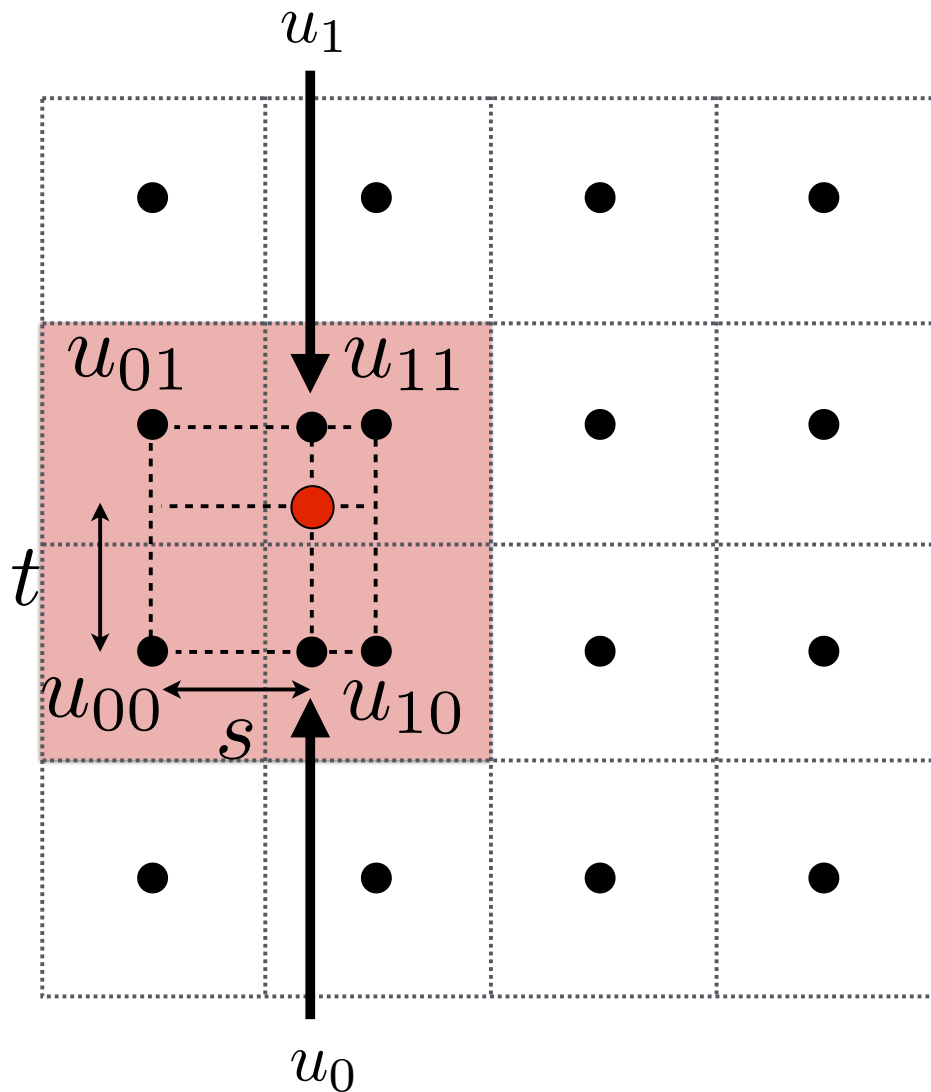
$$\text{lerp}(x, v_0, v_1) = v_0 + x(v_1 - v_0)$$

Two helper lerps (horizontal)

$$u_0 = \text{lerp}(s, u_{00}, u_{10})$$

$$u_1 = \text{lerp}(s, u_{01}, u_{11})$$

Bilinear Interpolation



Linear interpolation (1D)

$$\text{lerp}(x, v_0, v_1) = v_0 + x(v_1 - v_0)$$

Two helper lerps

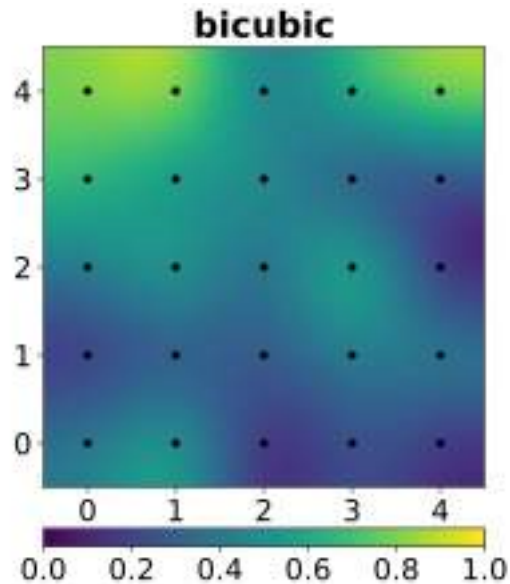
$$u_0 = \text{lerp}(s, u_{00}, u_{10})$$

$$u_1 = \text{lerp}(s, u_{01}, u_{11})$$

Final vertical lerp, to get result:

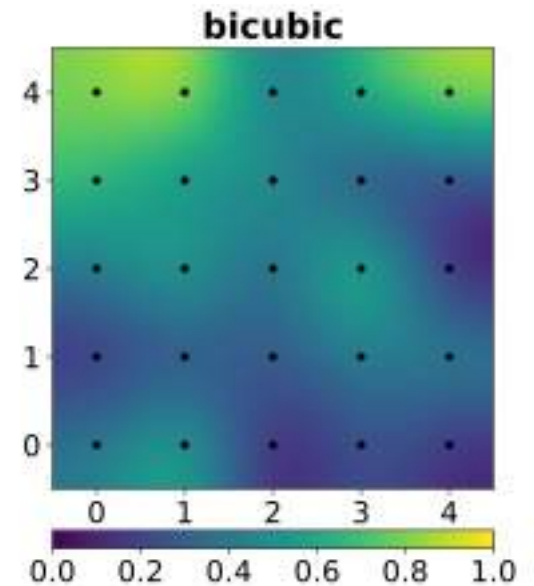
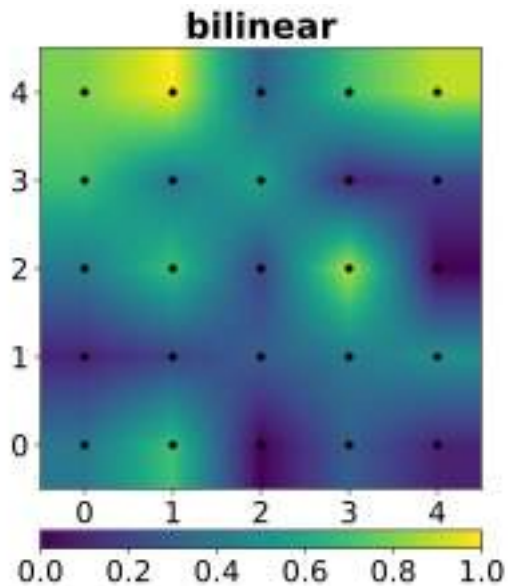
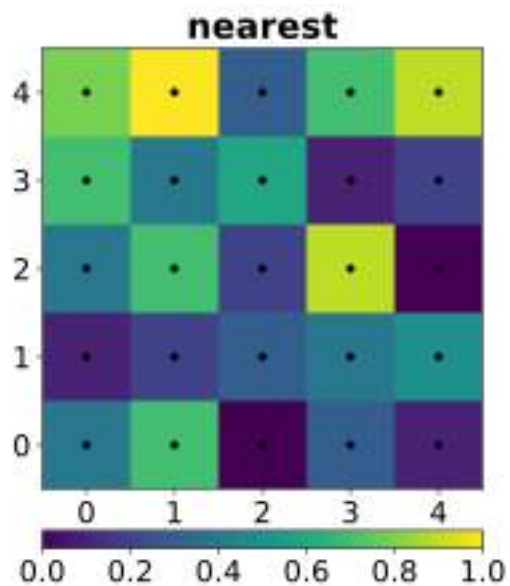
$$f(x, y) = \text{lerp}(t, u_0, u_1)$$

Bicubic Interpolation



$$p(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j.$$

Comparison



Comparison

Generally bilinear is good enough



Nearest



Bilinear



Bicubic

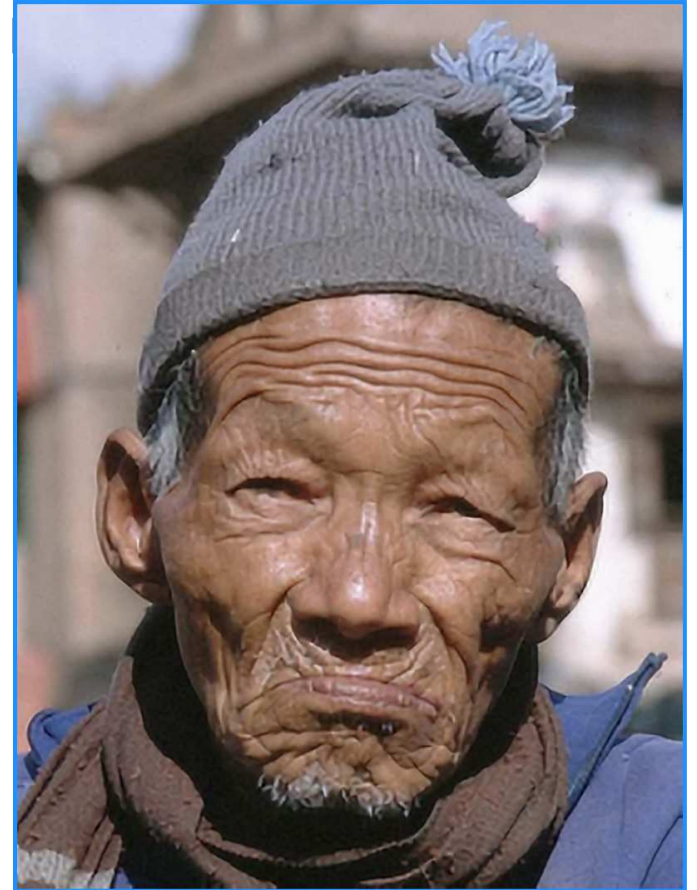
Super-Resolution



Original



Bi-Cubic



Super-Resolution

How to change aspect ratio?



Challenge



Changing aspect ratio causes distortion



Cropping may remove important contents

Seam Carving for Content-Aware Image Resizing

Shai Avidan

Mitsubishi Electric Research Labs

Ariel Shamir

The Interdisciplinary Center & MERL

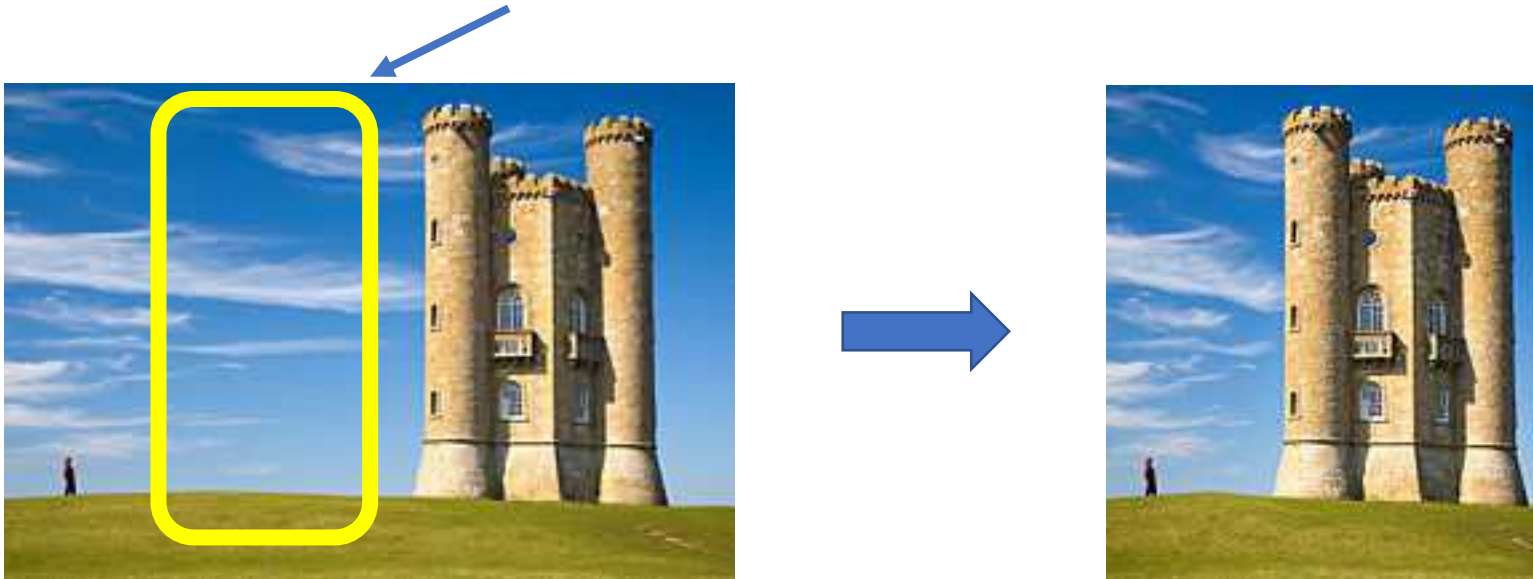


Content-aware resizing

Basic idea

Problem statement: we need to remove n pixels from each row

Basic idea: remove unimportant pixels

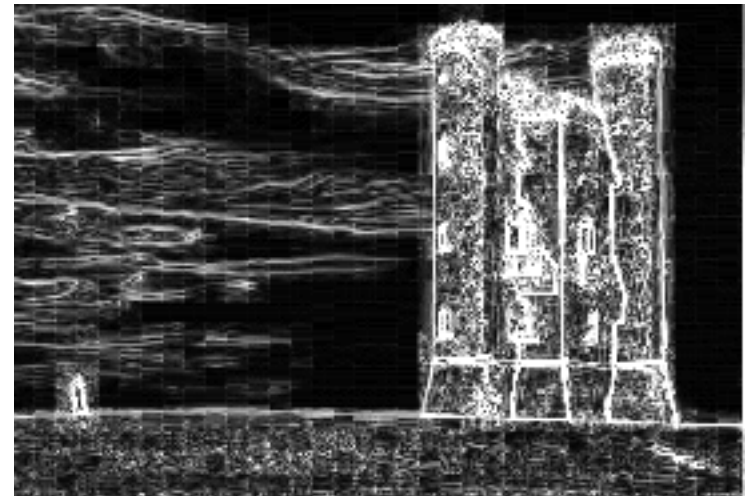
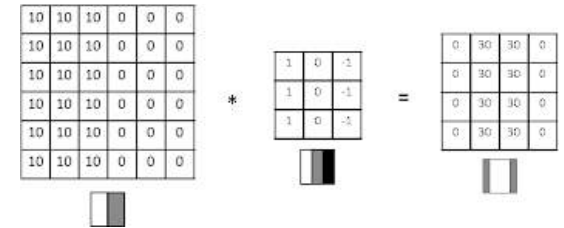


Importance of pixel

How to measure importance of a pixel?

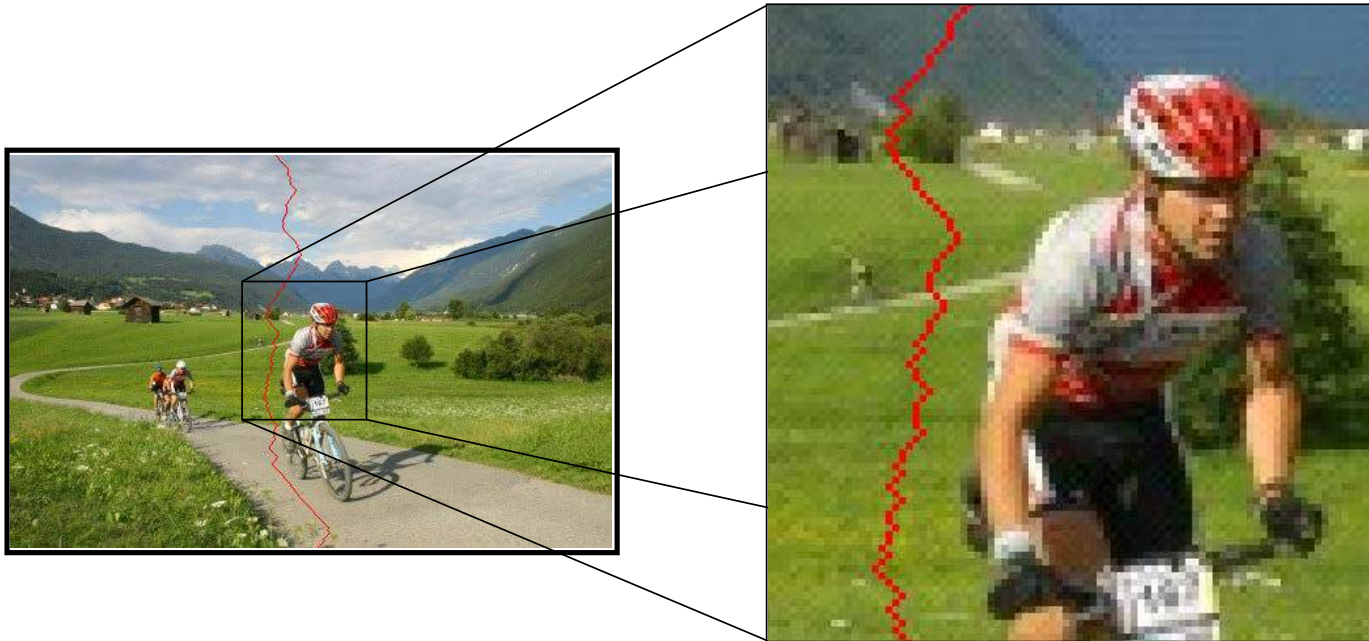
- A simple idea – edges are important
- Edge energy:

$$E(I) = \left| \frac{\partial I}{\partial x} \right| + \left| \frac{\partial I}{\partial y} \right|$$



Seam carving

Find connected path of pixels from top to bottom of which the edge energy is minimal



Finding the seam?



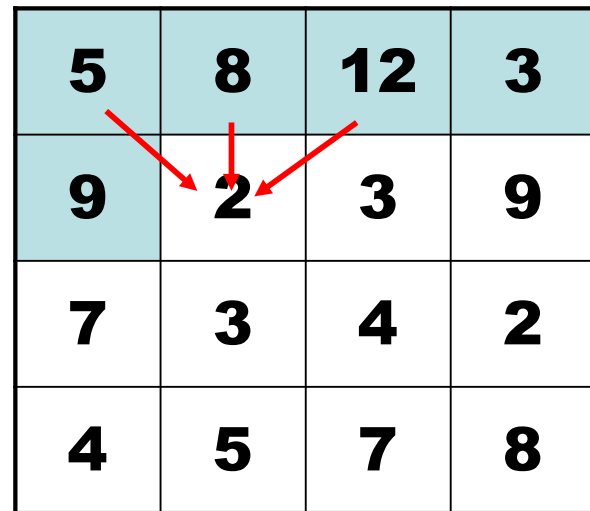
Finding the seam?

Going from top to bottom

- If $M(i,j)$ = minimal energy of a seam going through (i,j)
- Then:

$$M(i, j) = E(i, j) + \min(M(i-1, j-1), M(i-1, j), M(i-1, j+1))$$

- Solved by dynamic programming



5	8	12	3
9	2	3	9
7	3	4	2
4	5	7	8

Results



Original



Seam Carving



Scaling

Results



Cropping



Seams



Scaling

Can we enlarge an image?



Seam insertion

Find k seams to insert

Then interpolate pixels

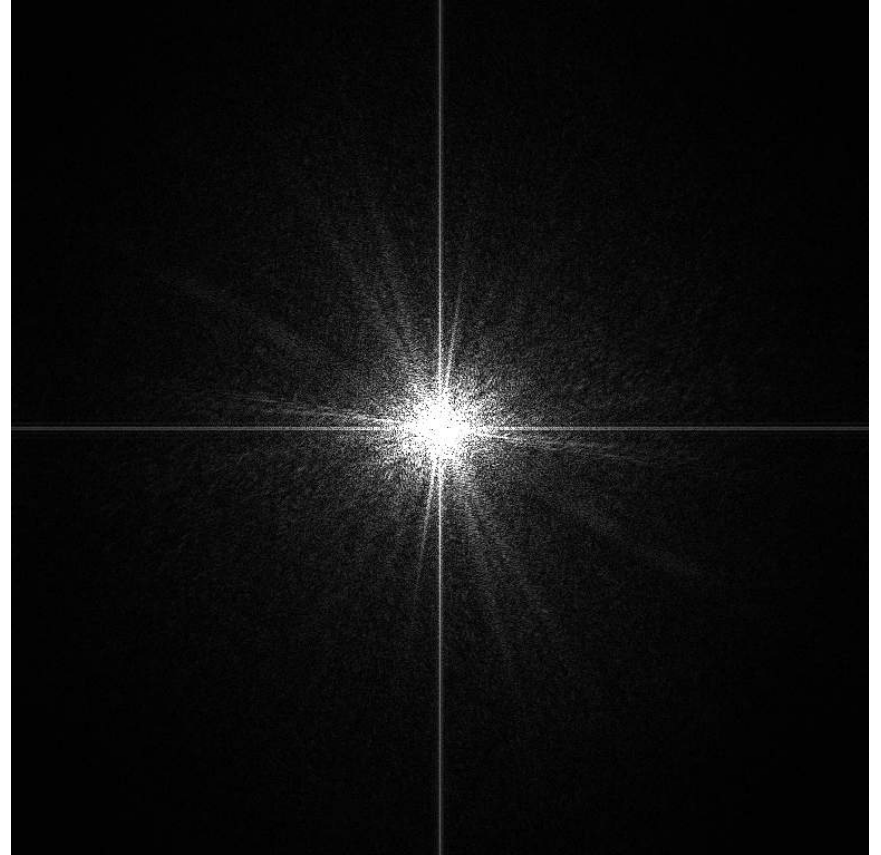


Questions?

Visualizing the frequency content of images



Spatial domain result

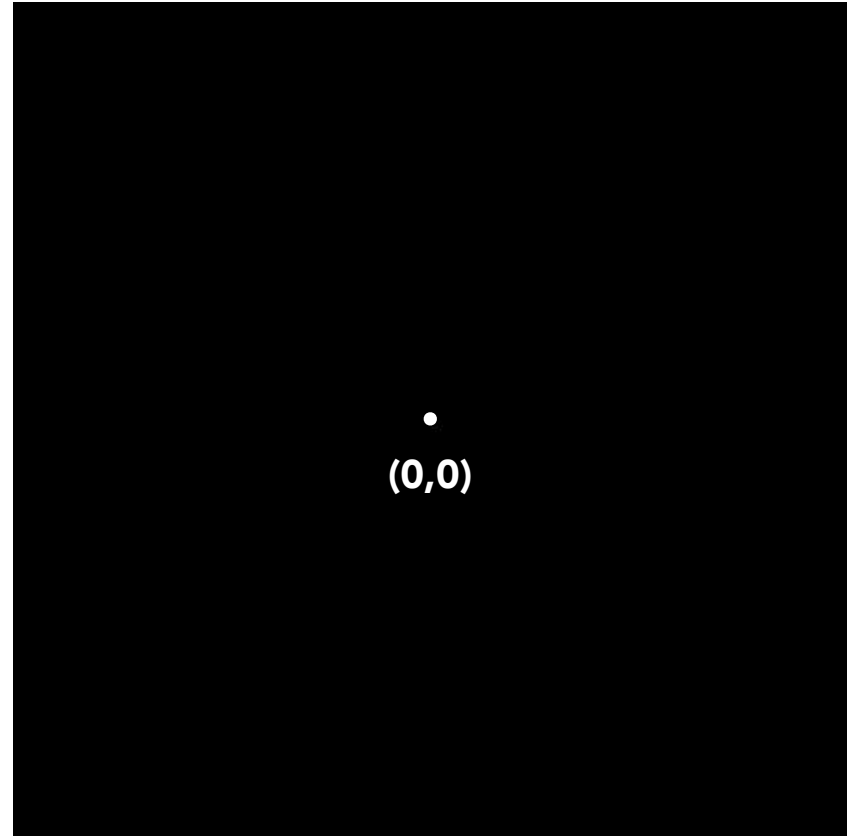


Spectrum

Constant signal

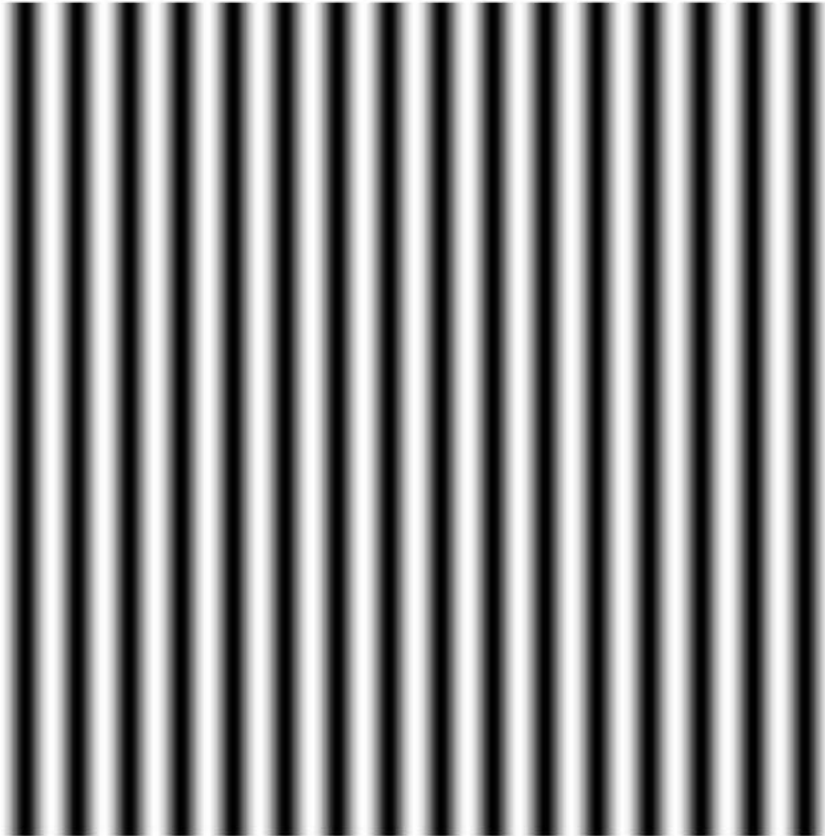


Spatial domain

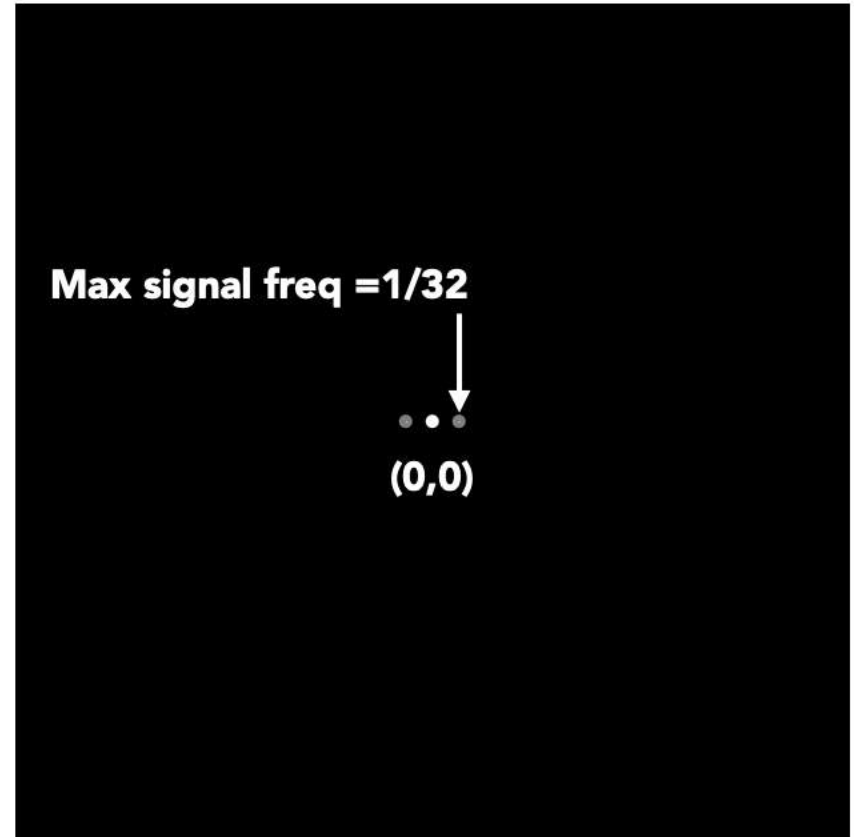


Frequency domain

$\sin(2\pi/32)x$ — **frequency 1/32; 32 pixels per cycle**

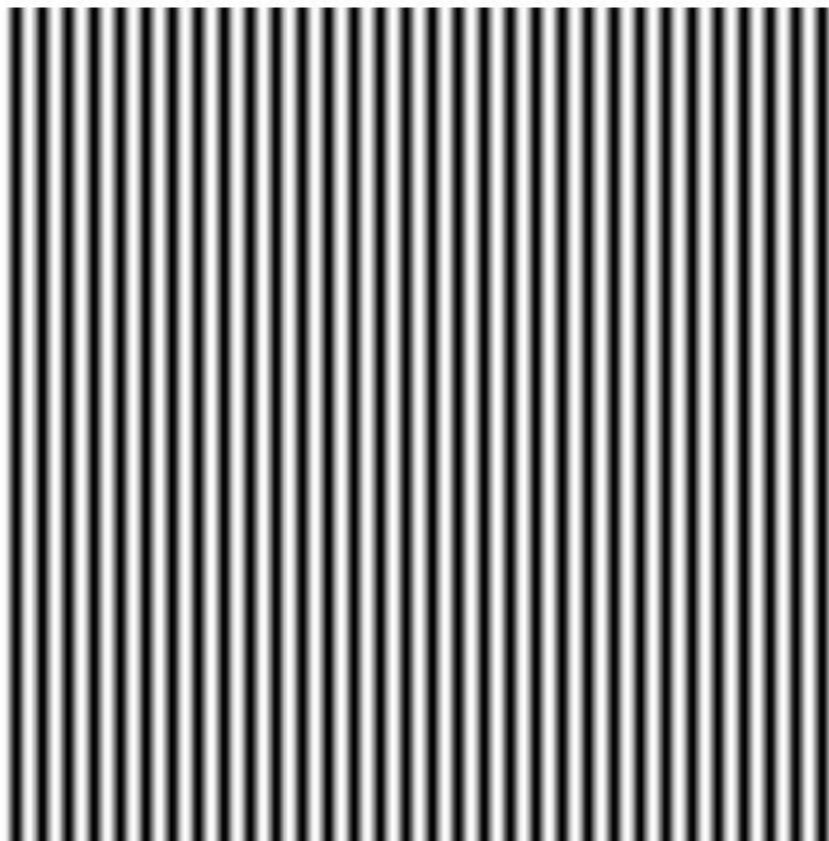


Spatial domain

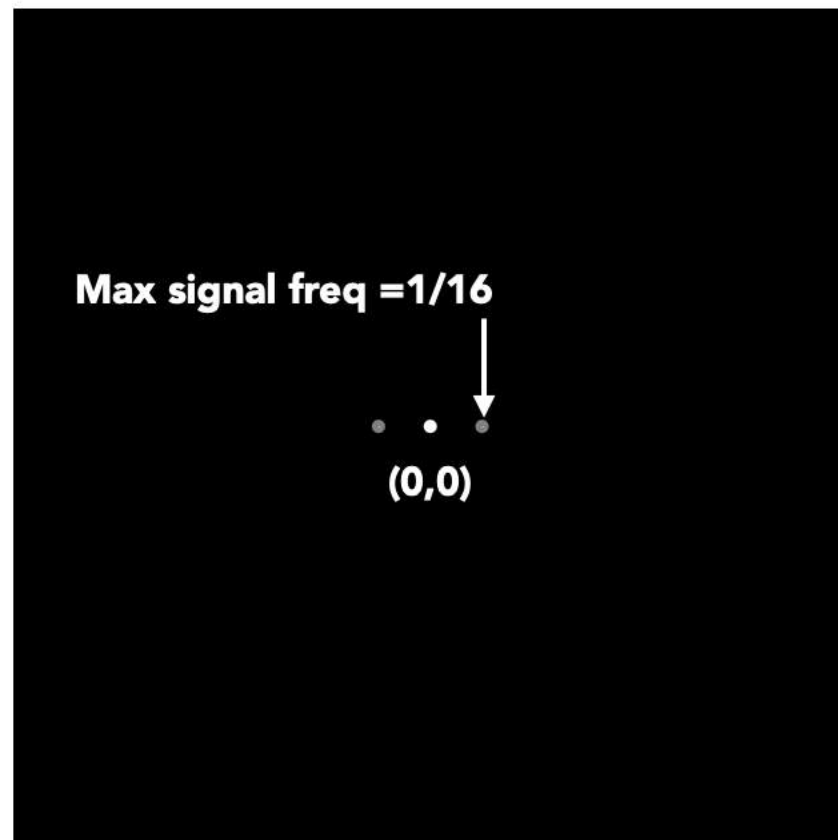


Frequency domain

$\sin(2\pi/16)x$ — **frequency 1/16; 16 pixels per cycle**



Spatial domain

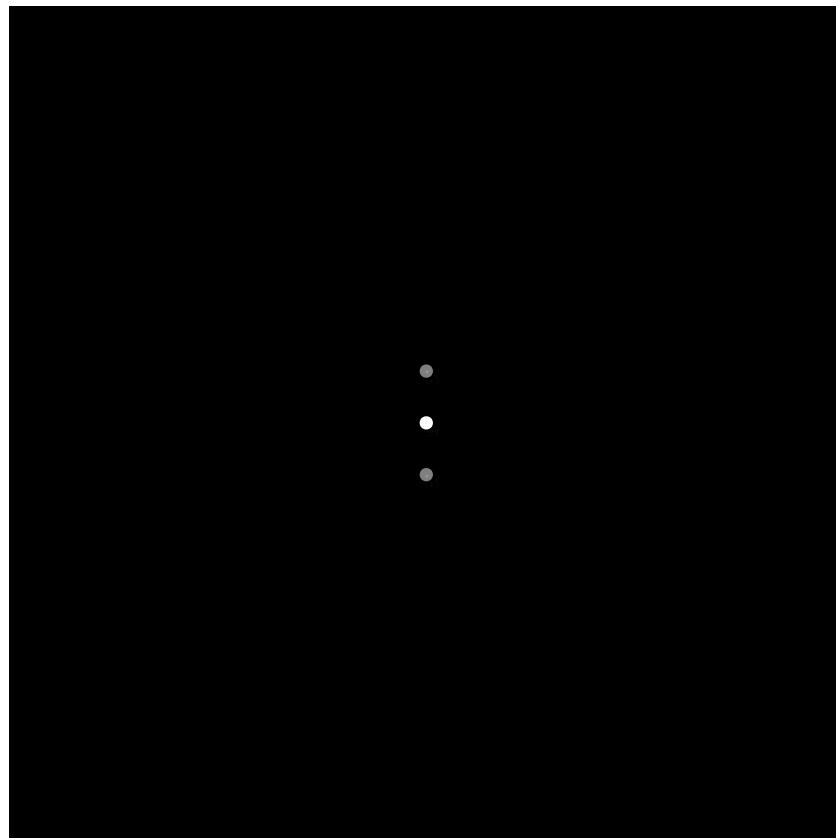


Frequency domain

$$\sin(2\pi/16)y$$

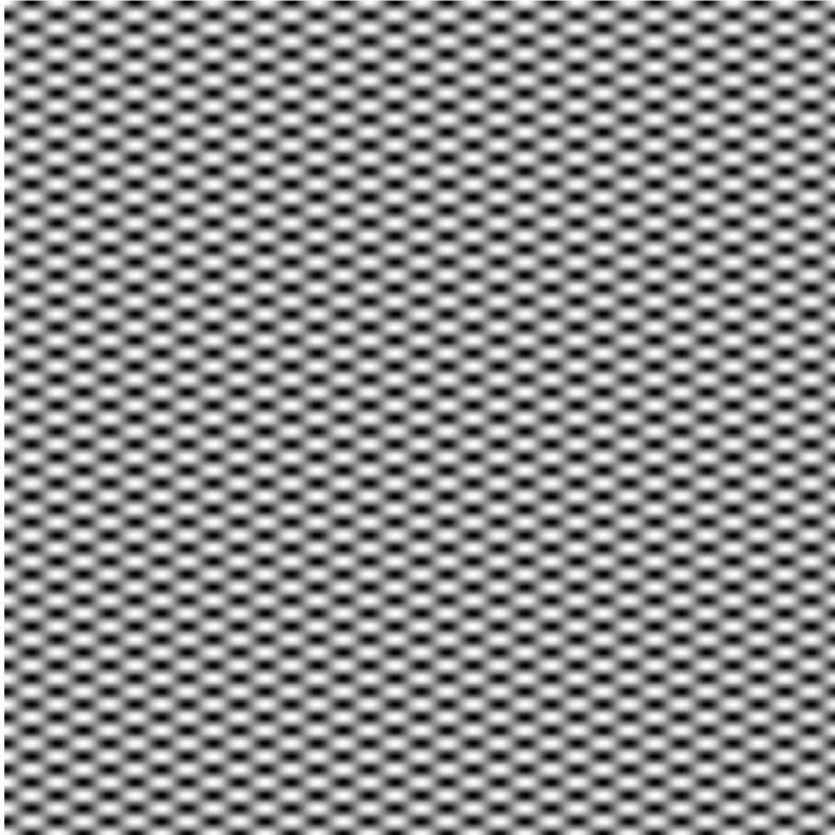


Spatial domain

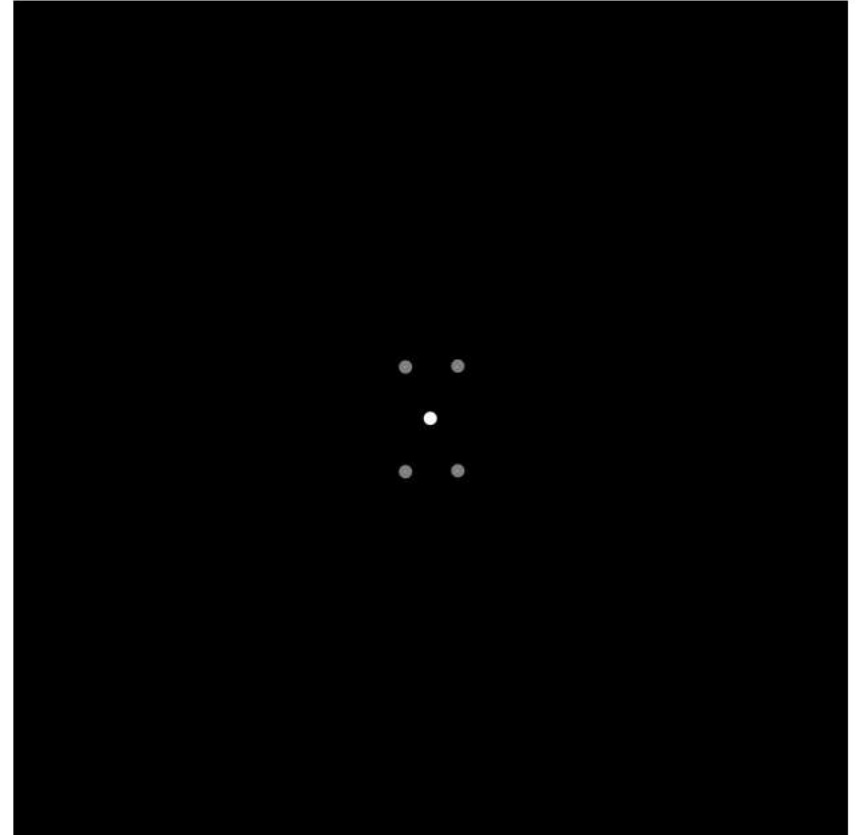


Frequency domain

$$\sin(2\pi/32)x \times \sin(2\pi/16)y$$



Spatial domain

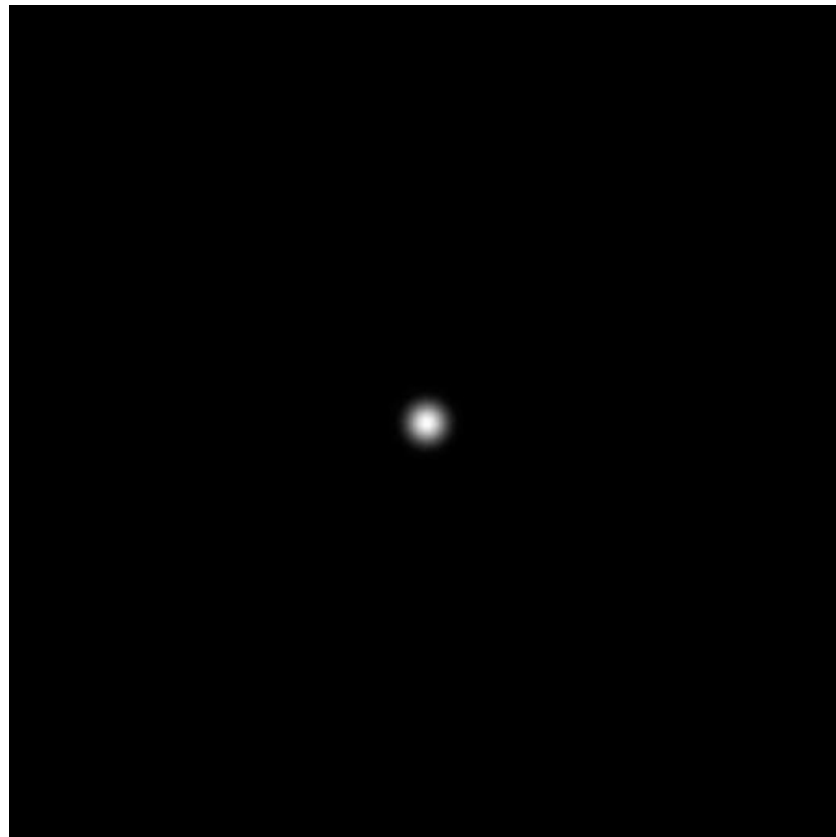


Frequency domain

$$\exp(-r^2/16^2)$$

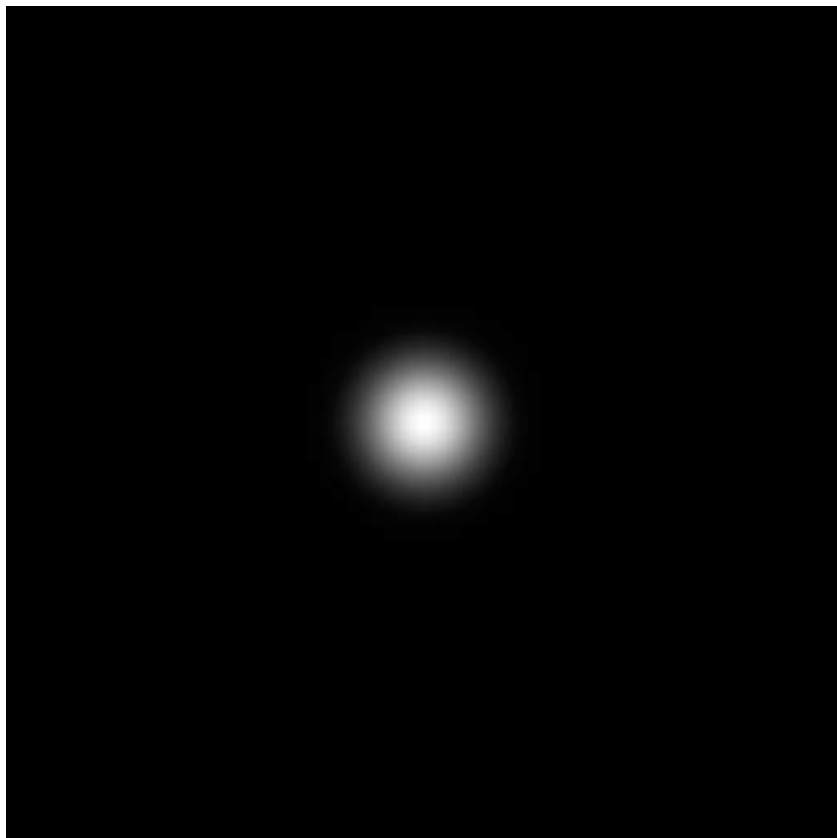


Spatial domain

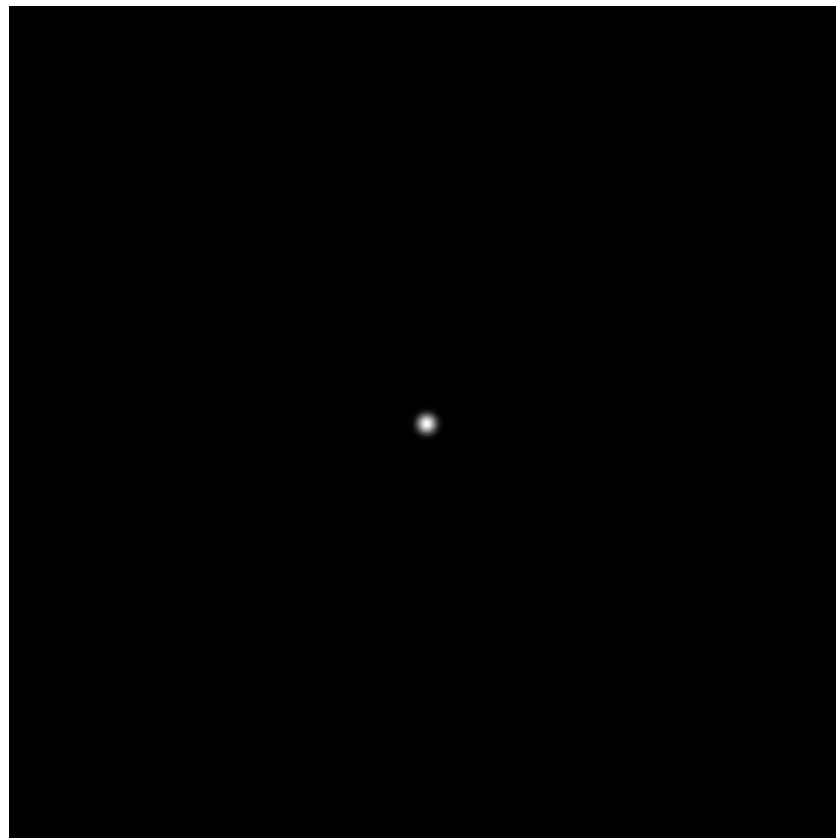


Frequency domain

$$\exp(-r^2/32^2)$$

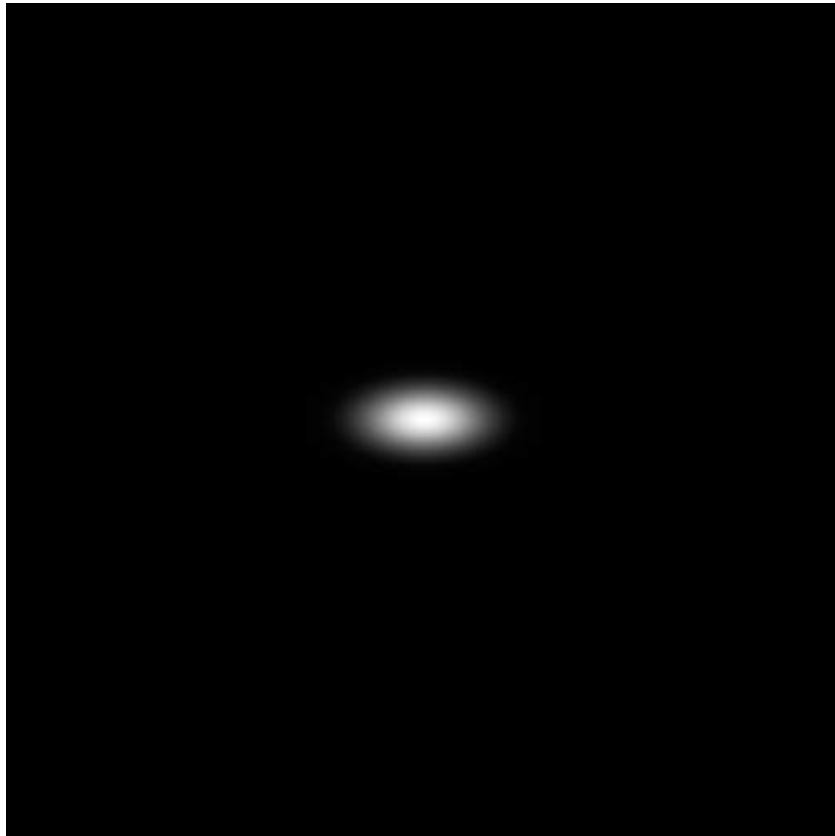


Spatial domain

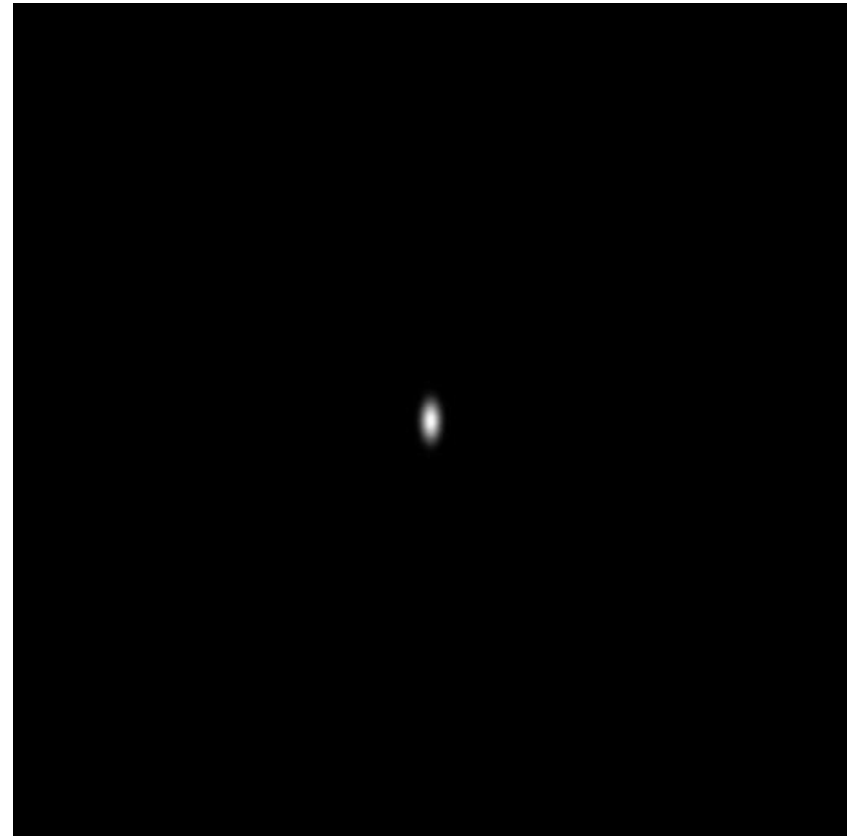


Frequency domain

$$\exp(-x^2/32^2) \times \exp(-y^2/16^2)$$

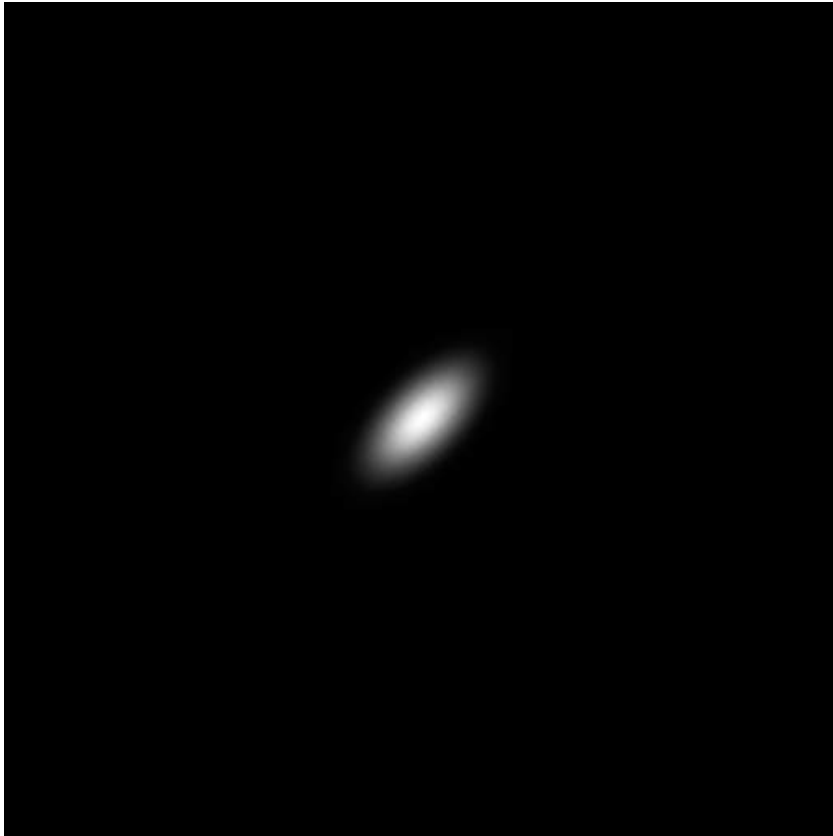


Spatial domain



Frequency domain

Rotate 45 $\exp(-x^2/32^2) \times \exp(-y^2/16^2)$



Spatial domain



Frequency domain